History-Deterministic Büchi Automata are Succinct

Antonio Casares
RPTU University Kaiserslautern
Landau
Germany

Aditya Prakash
Université Aix-Marseille
France

K. S. Thejaswini
Université Libre de Bruxelles
Belgium

Abstract

We describe a history-deterministic Büchi automaton that has strictly less states than every language-equivalent deterministic Büchi automaton. This solves a problem that had been open since the introduction of history-determinism and actively investigated for over a decade.

Our example automaton has 65 states, and proving its succinctness requires the combination of theoretical insights together with the aid of computers.

CCS Concepts

• **Theory of computation → Automata over infinite objects**.

Keywords

History deterministic automata, Succinctness, Büchi automata

1 Introduction

Automata over infinite words are a well-established tool with applications in the verification and synthesis of non-terminating systems [16, 22]. A major bottleneck in these applications is the exponential cost of determinising automata over infinite words. To circumvent this, recent research has focused on history-deterministic automata (HD automata hereafter): a class of "mildly" nondeterministic automata that offer the algorithmic benefits of determinism without the full cost of determinisation. Formally, an automaton is history-deterministic if there is a strategy resolving the nondeterministic choices on-the-fly without guessing the future. HD automata were introduced by Henzinger and Piterman [19] under the name of good-for-games automata, as they are exactly the class of automata that can be composed with infinite duration games on graphs while preserving the winner. This is the key property that enables their use in verification and synthesis (see e.g. [5, Page 22]). Equivalent definitions of history-determinism appear in the work of Kupferman, Safra, and Vardi [23] and were unified by Boker, Kuperberg, Kupferman, and Skrzypczak [4]. Moreover, Colcombet independently defined the concept of history-determinism in the setting of cost automata [8].

In this work, we focus on Büchi and coBüchi automata. A Büchi automaton is an automaton with some subset of its transitions marked as *significant*. A word is accepted by a Büchi automaton if it admits a run that contains infinitely many significant transitions. CoBüchi automata are the dual of Büchi automata: a word is accepted if it admits a run which contains only finitely many significant transitions.

We exemplify the above notions in the next example.

Example 1.1. The automaton described in Fig. 1—a version of [4, Fig.2]—accepts the language of words of the form $(xa+xb)^\omega$ which contain at least one of the strings $xaxa$ or $xbxb$ infinitely often.

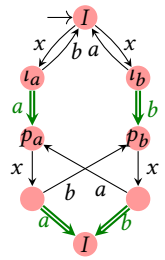


Figure 1: A HD Büchi automaton that is not determinisable by pruning. States labelled I are identified as the same state. Double green arrows represent significant transitions.

This automaton is HD. The only non-deterministic choice is on I over the letter x. A strategy resolving this non-determinism is simple: we alternate between choosing the transitions $I \xrightarrow{x} \iota_a$ and $I \xrightarrow{x} \iota_b$. If the input word is in the language, it must eventually contain $xaxa$ or $xbxb$, allowing us to progress to states p_a or p_b and visit an accepting transition.

As mentionned before, HD automata constitute an alternative for the use of deterministic automata in applications. In many procedures, such as LTL-synthesis, the main bottleneck is the size of the resulting deterministic automata [17]. One motivation for the use of HD automata is the potential reduction in size. For this reason, a central theme in the study of HD automata is *succinctness*: can HD automata be smaller than their deterministic counterparts?

For coBüchi automata, the landscape is well-understood. We know that HD co-Büchi automata can be exponentially smaller than the smallest equivalent deterministic ones [21]. Moreover, Abu Radi and Kupferman showed that HD coBüchi automata (with transition-based acceptance) can be minimised in polynomial time, leading to a canonical form for languages recognised by co-Büchi automata [1]. These theoretical results have subsequently enabled a canonical representation of ω-regular languages [15], algorithms for learning HD coBüchi automata [25], and practical implementation efforts [13, 14].

In contrast, the situation for Büchi automata has remained elusive. While it is known that any HD Büchi automaton admits an equivalent determinised Büchi automaton of quadratic size [21], it is unknown whether this bound could be matched or improved. Actually, it was not even known if any HD Büchi automaton admits an equivalent determinised Büchi automaton of the same size. In this work, we tackle this question:

Is there an HD Büchi automaton that is strictly smaller than any equivalent deterministic Büchi automaton?

In fact, for nearly a decade, it was not known whether HD Büchi automata could simply be determinised by pruning transitions (as

is the case for automata over finite words). This was settled in the negative by Boker et al. [4] with the help of the HD Büchi automaton with 7 states discussed in the example above. However, there is an equivalent deterministic automaton with only 4 states (see Figure 3a). The computational complexity of the determinisation of HD Büchi automata was only recently proved to be in polynomial time by two independent works [3, 28].

In recent years, the research community working on HD automata has been very active, and recent results—for example the works by Abu Radi and Kupferman [1] or by Lehtinen and Prakash [24]—have gained a lot of insight on the structure of HD automata. The fact that this question remained open points to a gap of our understanding of the structure of HD and deterministic Büchi automata.

Several works raise the above question on the succinctness of HD Büchi automata. A conjecture stating that all HD parity automata (and in particular HD Büchi automata) are determinisable by prunning was stated by Colcombet in 2012 [9, Conj. 8]. The succinctness question was asked in 2013 [4] (for HD automata with various acceptance conditions) and solved in 2015 for coBüchi automata [21]. More recently, this question has been discussed in various works [1, 3, 28]. For the subclass of Muller languages (languages that can be described by a Boolean condition on the letters that appear infinitely often), it has been shown that HD parity and Büchi automata are not more succinct than deterministic ones [6, Cor. 4.16], while HD Rabin automata can be exponentially smaller than deterministic ones [7, Thm. 21]. In his PhD thesis, Prakash states a conjecture implying that any HD Büchi automaton admits an equivalent deterministic one of the same size [26, Conj 2].

In this paper, we answer the succinctness question positively.

Theorem 1.2. *There is a history-deterministic Büchi automaton that has strictly fewer states than any equivalent deterministic Büchi automaton.*

Concretely, we provide an HD Büchi automaton with 65 states and prove that any equivalent deterministic automaton requires at least 66 states.

Verifying this gap is non-trivial. Although tools like SPOT exist for minimising deterministic transition-based Büchi automata [11], they use SAT solvers. Our attempt to feed the full 66-state deterministic automaton to such a solver was perhaps overly optimistic; the resulting CNF formula grew large enough to exhaust our memory long before it exhausted the search space and to the best of our knowledge, the solver is still pondering over the existence of a 65-state deterministic automaton. Furthermore, the minimisation of deterministic Büchi automata is NP-complete for both state-based acceptance [29] or transition-based acceptance [27].

Hence, we construct our witness automaton in such a way that most of the states of the automaton are essential and cannot be minimised. This is done by leveraging structural insights regarding HD co-Büchi automata, drawing primarily on the work of Abu Radi and Kupferman [1]. We use the characterisation of structure of minimal HD co-Büchi automata to reason about the minimality of the complement of our HD Büchi automaton.

Even with these theoretical reductions, we are left to prove that the smallest DFA separating two specific languages on finite words requires at least 5 states. To establish this lower bound, we employ the learner tool DFAMiner [10]. The tool formulates the existence of a 4-state separator as a SAT instance then proves its unsatisfiability, thereby making our proof computer-aided.

Organisation. After introducing the necessary preliminaries in Section 2, we give some intuition and first steps towards the construction of the automaton in Section 3. This intuition is given by considering some conjectures and constructing automata to disprove them. This section serves two purposes:

- First, it allows to give a more gentle introduction to the general example and the techniques to prove that it is history-deterministic.
- Secondly, it provides some insights on what are some necessary properties that succinct HD Büchi automata must satisfy. This explains why small succinct automata are challenging to find.

We describe our full 65-state example in Section 4, by combining the ideas from Section 3. Finally, in Section 5 we present the proof of succinctness for Theorem 1.2; in this section we also include encodings of the automata over finite words that we use for DFAminer.

2 Preliminaries

For a natural number i, we use $[i]$ to denote the set $\{1, 2, \ldots, i\}$.

2.1 Automata and history-determinism

An *alphabet*, often denoted by Σ, is a finite set.

Automata. A nondeterministic Büchi (resp. coBüchi) automaton $\mathcal{A} = (Q, \Sigma, \Delta, q_0, F)$ consists of a finite set of *states* Q, an *initial state* $q_0 \in Q$, a set of labelled *transitions* $\Delta \subseteq Q \times \Sigma \times Q$ and a subset $F \subseteq \Delta$ of *significant* transitions. We denote transitions (p, a, q) by $p \xrightarrow{a} q$, and transitions (p, a, q) that are significant by $p \overset{a}{\Rightarrow} q$.

Runs. A *run* of $\mathcal{A}$ over an infinite word w in Σ^ω is an infinite path ρ of the automaton starting at q_0 such that the edges of ρ are labelled by the letters of w in sequence. If $\mathcal{A}$ is a Büchi automaton, then ρ is an *accepting run* if ρ contains infinitely many significant transitions. Dually, if $\mathcal{A}$ is a coBüchi automaton, then ρ is an *accepting run* if ρ contains finitely many significant transitions. Infinite runs that are not accepting are dubbed *rejecting runs*.

In the rest of the paper, the word automaton means either a nondeterministic Büchi automaton or a nondeterministic coBüchi automaton, unless mentioned otherwise. A *subautomaton* of $\mathcal{A}$ is an automaton obtained by removing transitions of $\mathcal{A}$.

Languages. A word w is *accepted* by an automaton $\mathcal{A}$ if there is an accepting run of $\mathcal{A}$ on w, and *rejected* otherwise. The *language* of $\mathcal{A}$ is the set of words that are accepted by $\mathcal{A}$, which we denote as $L(\mathcal{A})$. Two automata $\mathcal{A}$ and $\mathcal{B}$ are *language-equivalent* if they have the same language.

For a state q in $\mathcal{A}$, we write $(\mathcal{A}, q)$ to denote the automaton where we set q to be the initial state. We say that two states p, q of $\mathcal{A}$ are *language-equivalent* if $L(\mathcal{A}, p) = L(\mathcal{A}, q)$.

Determinism. We say that an automaton is *deterministic* if, from every state and for every letter, there is at most one outgoing transition from that state on that letter.

History determinism. A *resolver* for an automaton $\mathcal{A}$ is a function of type $\Delta^* \times \Sigma \to \Delta$ that takes as input a finite run in $\mathcal{A}$ and a letter

a, and outputs a transition on a from the endpoint of the run. This way, a *resolver induces* a run on every word in Σ^*. We say that a resolver for $\mathcal{A}$ is an *HD-resolver* if it induces an accepting run on every word in the language. We say that an automaton is *history deterministic*, HD for short, if there is an HD-resolver for it.

We refer to Example 1.1 for an example of an HD automaton. We give next a non-example.

Example 2.1. In Figure 2 we show a Büchi automaton that is not history-deterministic. This automaton accepts the words that eventually only contain the letter a. However, to resolve the non-determinism on letter a, the automaton needs to guess the future. For every strategy deciding to go to the state on the right after a finite prefix, there is an extension of the form ba^ω so that this strategy would reject a word that is in the language.

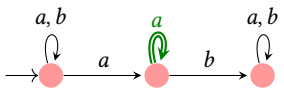


Figure 2: A Büchi automaton that is not HD.

Interpretation of figures. Throughout the paper, we include many figures, some of them representing automata with more states we would wish to. To represent these automata, we use some shorthands. First, a state may appear several times in a figure; the final automaton is the result of gluing together the states with the same name. When many states have the same target over the same letter σ, we may surround these states by a box, and use transitions outgoing the box. A transition on σ from a box to a state q means that from all states of the box, the σ-transition goes to q. As in the text, double green arrows represent significant transitions for Büchi automata. (For coBüchi automata, significant transitions will be double arrows, but red. These rarely occur.) Sometimes we use different colours and shapes or sizes for states; this is mostly for aesthetic purposes (in Figures 5, 6, and 7, colour differentiation also serves to identify language-equivalent states).

2.2 Structure of HD Büchi automata

The quadratic determinisation construction for HD Büchi automata of Kuperberg and Skrzypczak [21] first involved proving that every HD Büchi automaton can be simplified so that it satisfies certain properties. Their analysis was then refined later by Acharya, Jurdziński, and Prakash [3]; we will use the presentation from Prakash's PhD thesis [26] in this work. We begin by defining notions that are useful towards this.

Simulation relation. Simulation is a relation between two automata that is coarser than language inclusion, that is, if $\mathcal{B}$ simulates $\mathcal{A}$, then $L(\mathcal{A}) \subseteq L(\mathcal{B})$ [18]. Concretely, the simulation relation can be described as a game between two players Adam and Eve, called the simulation game. The *simulation game of automaton $\mathcal{A}$ by automaton $\mathcal{B}$* starts with an Eve's token at the initial state of the automaton $\mathcal{B}$ and Adam's token at the initial state of $\mathcal{A}$. In each round,

(1) Adam selects a letter and moves his token along a transition on that letter in $\mathcal{A}$,
(2) Eve moves her token along a transition on that letter in automaton $\mathcal{B}$.

Consequently, in the limit of such a play, Adam constructs a run on his token in $\mathcal{A}$, and Eve a run on her token in $\mathcal{B}$, both on the same word chosen by Adam. Eve *wins* if either Eve's run is accepting or Adam's run is rejecting. If Eve has a winning strategy in this game, then we say that automaton $\mathcal{B}$ *simulates* automaton $\mathcal{A}$. For two states p and q in an automaton $\mathcal{A}$, we will often write q simulates p in $\mathcal{A}$ to say that $(\mathcal{A}, q)$ simulates $(\mathcal{A}, p)$.

Reach covering. A *reachability automaton* is a Büchi automaton where all of its significant transitions lead to an *accepting sink state* that has a significant self-loop on every letter.

For every Büchi automaton $\mathcal{A}$, we use $\texttt{reach}(\mathcal{A})$ to denote the reachability automaton that is obtained by redirecting all the significant transitions of $\mathcal{A}$ to an accepting sink-state, and by not modifying all the transitions that are not significant.

We say that q is a *reach-deterministic state* in $\mathcal{A}$ if the accessible portion of $(\texttt{reach}(\mathcal{A}), q)$ is *deterministic*, i.e., every word w has an unique run from q in $\texttt{reach}(\mathcal{A})$.

We say that $\mathcal{A}$ has *reach-covering* if for every state p in $\mathcal{A}$, there is a reach-deterministic state q of $\mathcal{A}$ such that the following two conditions hold.

(1) p and q are language-equivalent in $\mathcal{A}$.
(2) p simulates q in $\mathcal{A}$.

Note that, since reach-deterministic states simulate themselves, $\mathcal{A}$ has reach-covering if and only if the above conditions hold for states p that are not reach-deterministic.

Semantic determinism. We say that an automaton $\mathcal{A}$ is *semantically deterministic*, SD for short, if outgoing transitions on the same letter from language-equivalent states lead to language-equivalent states. That is, if p, q are language-equivalent states in $\mathcal{A}$ and $p \xrightarrow{a} p', q \xrightarrow{a} q'$ are transitions on the same letter a, then p' and q' are also language-equivalent.

Normal form. Given a Büchi automaton $\mathcal{A}$, consider the graph consisting only of non-significant transitions. The automaton $\mathcal{A}$ is *normal* if every non-significant transition is in some SCC of this graph. We note that every Büchi automaton can be converted into a language-equivalent normal automaton, by changing all non-significant transitions of $\mathcal{A}$ changing of SCC to be significant; this transformation does not introduce any accepting cycles, and thus, does not increase the language of the automaton.

Simplified Büchi automata. We say that a Büchi automaton $\mathcal{A}$ is *simplified* if $\mathcal{A}$ is semantically-deterministic, normal, and has *reach-covering*. Acharya, Jurdziński, and Prakash showed that every HD Büchi automaton can be *simplified* in polynomial time without increasing the number of state [26, Lemma 5.8]. We will use their result that every simplified Büchi automaton is history-deterministic as well, in order to prove history-determinism of various Büchi automata.

Lemma 2.2 ([26, Lemma 5.7])**.** *Every simplified Büchi automaton is history-deterministic.*

Proof sketch. The HD-resolver for a simplified Büchi automaton $\mathcal{A}$, when at state p, simulates an auxiliary run from a reach-deterministic state q such that p and q are language-equivalent in $\mathcal{A}$ and p simulates q in $\mathtt{reach}(\mathcal{A})$. Note that the auxiliary run from the reach-deterministic state q is deterministic. When the auxiliary run reaches the accepting sink state in $\mathtt{reach}(\mathcal{A})$, we know that since this auxiliary run was being simulated by the resolver in $\mathtt{reach}(\mathcal{A})$, the run produced by the resolver must also contain a significant transition. When the resolver's run takes a significant transition to a state p', it starts simulating an auxiliary run from a good state q' that is language-equivalent to p' in $\mathcal{A}$ again. □

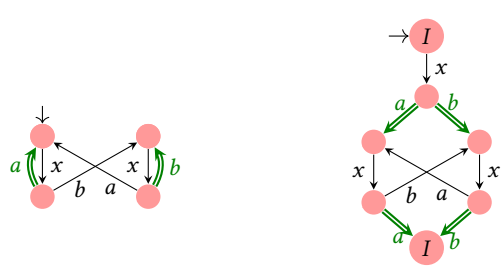
3 Graveyard of conjectures and the automata that killed them

We begin by analysing why naïve determinisation techniques fail. We provide increasingly complex counterexamples to specific "straw man" determinisation techniques, which then helps us build the intuition required for our main result. These examples demonstrate the pitfalls that our final construction must avoid. Although the definition of the 65-state automaton can be understood directly by skipping ahead to Section 4, we strongly recommend reading this section first to understand the method behind the madness of the final construction.

A motivating example. We start with the example automaton $\mathcal{A}_{\mathsf{BKKS}}$ from Figure 1. We show two deterministic Büchi automata that are language-equivalent to $\mathcal{A}_{\mathsf{BKKS}}$ and that have at most as many states in Fig. 3.

First, observe that the 4 bottom states of $\mathcal{A}_{\mathsf{BKKS}}$ are reach-deterministic, while the non-reach-deterministic states are $\{I, \iota_a, \iota_b\}$. We refer to these 3 states as the 'nondeterministic component' of $\mathcal{A}_{\mathsf{BKKS}}$.

The automaton on the left of Fig. 3 is obtained by *rewiring*: keeping the deterministic component and redirecting significat transitions. This is formally defined in Section 3.1. The second automaton, on the other hand, is obtained by replacing the 'nondeterministic component' of $\mathcal{A}$ with a deterministic part with the same number of states.



(a) Rewiring **(b) Replacing the nondeterministic component**

Figure 3: Two language-equivalent deterministic Büchi automata for the BKKS automaton.

We describe automata where these two determinisation techniques do not work (without strictly increasing the number of states) in Sections 3.1 and 3.3, respectively. We then present another determinisation tactic in Section 3.2. We then use the examples and insights from these section to motivate our succinct HD Büchi automaton in Section 4.

3.1 The Rewiring Conjectures

For a simplified Büchi automaton $\mathcal{A}$, a *rewiring* of $\mathcal{A}$ is another deterministic Büchi automaton $\mathcal{B}$ such that the following two conditions hold.

(1) The states of $\mathcal{B}$ are the reach-deterministic states of $\mathcal{A}$, and the non-significant transitions of $\mathcal{B}$ are the same as the non-significant transitions that are outgoing from the reach-deterministic states of $\mathcal{A}$.
(2) The significant transitions of $\mathcal{B}$ are of the form $p \overset{a}{\Rightarrow} q$, such that $L(\mathcal{A}, q) = a^{-1}L(\mathcal{A}, p)$.

In other words, $\mathcal{B}$ is obtained by redirecting the significant transitions from reach-deterministic states of $\mathcal{A}$ to other reach-deterministic states of $\mathcal{A}$, so that only reach-deterministic states are reached. Prakash [26, Conjecture 2] thus proposed what we call the *weak-rewiring conjecture*:

Conjecture 3.1 (Weak-rewiring conjecture)**.** *Every simplified HD Büchi automaton has a language-equivalent rewiring.*

Towards disproving the weak-rewiring conjecture, we first disprove a stronger version of the conjecture. The strong-rewiring conjecture states that we can redirect significant transitions towards reach-deterministic states, while preserving the language, one-by-one and in an arbitrary order. (In the weak conjecture, we are allowed to apply all rewirings at once, which may be necessary to preserve language-equivalence.)

Conjecture 3.2 (Strong-rewiring conjecture)**.** *Let $\mathcal{A}$ be a simplified HD Büchi automaton, and let p be a reach-deterministic state in $\mathcal{A}$ such that there is transition of the form $p \overset{a}{\Rightarrow} p'$ where p' is not a reach-deterministic state. Then, there exists a reach-deterministic state q in $\mathcal{A}$ such that $L(\mathcal{A}, q) = a^{-1}L(\mathcal{A}, p)$, and the automaton $\mathcal{B}$, obtained by removing all transitions of the form $p \xrightarrow{a} p''$ in $\mathcal{A}$ and adding the transition $p \overset{a}{\Rightarrow} q$, is language-equivalent to $\mathcal{A}$.*

3.1.1 Strong rewiring ☠. To disprove the strong-rewiring conjecture, we will consider the automaton $\mathcal{A}_{\mathtt{strong}}$ in Fig. 4, over the alphabet $\Sigma = \{a, b, c, x, y\}$.

We describe the language recognised by $\mathcal{A}_{\mathtt{strong}}$. Define $L_\alpha = (x^*y^*)^*x\alpha$, for each $\alpha \in \{a, b, c\}$, and let

$$L_{\alpha\beta\beta} = L_\alpha L_\beta L_\beta.$$

The language recognised by $\mathcal{A}_{\mathtt{strong}}$ in Fig. 4 consists of words containing infinitely often infixes in $L_{\alpha\beta\beta}$, for some $\alpha \neq \beta$, as well as containing infinitely many ys. Formally, the language $L(\mathcal{A}_{\mathtt{strong}})$ is the following:

$$L_{\mathtt{strong}} = (\Sigma^* \cdot \bigcup_{\alpha\neq\beta\in\{a,b,c\}} L_{\alpha\beta\beta} \cdot \Sigma^* \cdot y)^\omega.$$

To intuitively see that this automaton indeed recognises the language $L_{\mathtt{strong}}$, note that whenever an infix $L_\alpha L_\beta L_\beta$ is read from I, with $\alpha \neq \beta$, the automaton can produce a run

$$I \xrightarrow{L_\alpha} p_\alpha \xrightarrow{L_\beta} q_\beta \xrightarrow{L_\beta} Y.$$

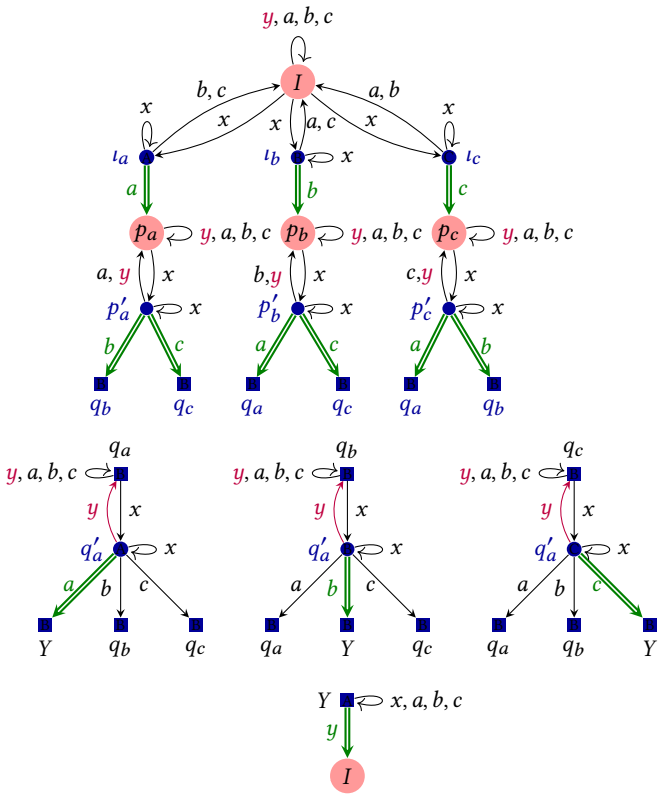


Figure 4: HD Büchi automaton $\mathcal{A}_{\mathtt{strong}}$ for which the strong-rewiring conjecture is false. To improve readability, we represent the automaton "by chunks"; note that some states appear multiple times (in total, the automaton has 17 states).

From Y, the automaton produces a significant transition and moves to I upon reading letter y.

Although not necessary for establishing Theorem 1.2, we provide full proofs of the next result, as this is a similar, but simpler, version of the proof of correctness for our main automaton in Section 4.

Lemma 3.3. *The automaton $\mathcal{A}_{\mathtt{strong}}$ recognises the language $L_{\mathtt{strong}}$. Moreover, it is history-deterministic and simplified.*

Proof. For the second claim, we need to show that $\mathcal{A}_{\mathtt{strong}}$ is semantically deterministic and has reach-covering (which implies HDness by Lemma 2.2).

$L(\mathcal{A}_{\mathtt{strong}}) = L_{\mathtt{strong}}$ *and semantic determinism.* First, we observe that $L_{\mathtt{strong}}$ is *prefix-independent*, i.e., for every letter $\sigma \in \Sigma$, $\sigma^{-1}L = L$. We show that for all states q in $\mathcal{A}_{\mathtt{strong}}$, $L(\mathcal{A}_{\mathtt{strong}}, q) = L_{\mathtt{strong}}$.

Let w be a word in $L_{\mathtt{strong}}$. We build an accepting run from q on w. Since w contains infinitely many y's, an arbitrary run from q eventually reaches one of the states $I, p_a, p_b, p_c, q_a, q_b, q_c$ (after reading a y). Furthermore, observe that we have the following language-equalities.

$$L(\mathtt{reach}(\mathcal{A}_{\mathtt{strong}}), I) = L_a \cup L_b \cup L_c$$
$$L(\mathtt{reach}(\mathcal{A}_{\mathtt{strong}}), p_\alpha) = \bigcup_{\beta \neq \alpha} L_\beta$$
$$L(\mathtt{reach}(\mathcal{A}_{\mathtt{strong}}), q_\alpha) = L_\alpha \cup \bigcup_{\beta \neq \alpha} L_\beta L_\beta$$

Due to the above equations, every word w in $L(\mathcal{A}_{\mathtt{strong}})$ and every state q in the set $\{I, p_a, p_b, p_c, q_a, q_b, q_c\}$ is such that some prefix of w is in $L(\mathtt{reach}(\mathcal{A}_{\mathtt{strong}}), q)$. Thus, there is a finite run from q on w in $\mathcal{A}_{\mathtt{strong}}$ that contains a significant transition. It follows by induction and the prefix independence of L that there is a run containing infinitely many significant transitions from every state of $\mathcal{A}_{\mathtt{strong}}$ on every word in L, as desired.

$\mathcal{A}_{\mathtt{strong}}$ *has reach covering.* Observe that every state in $\mathcal{A}_{\mathtt{strong}}$ apart from $I, \iota_a, \iota_b, \iota_c$ is reach-deterministic. We show that

(1) I simulates q_a in $\mathtt{reach}(\mathcal{A}_{\mathtt{strong}})$, and
(2) ι_α simulates q'_α in $\mathtt{reach}(\mathcal{A}_{\mathtt{strong}})$, for each $\alpha \in \{a, b, c\}$.

This implies reach covering for $\mathcal{A}_{\mathtt{strong}}$.

To see this, note that the only nondeterministic choice appears on state I, over the letter x. Eve has the following strategy to win the simulation-game: If Adam's token is in q_α and Adam gives the letter x, then Eve takes the transition on x to ι_α. If Eve uses this strategy and the simulation game in $\mathtt{reach}(\mathcal{A}_{\mathtt{strong}})$ starts from the starting positions (q_a, I) or $(q'_\alpha, \iota_\alpha)$, then

(1) whenever Eve's state is in I, Adam's state is q_α for some α in $\{a, b, c\}$, and
(2) Eve's state is in ι_α if and only if Adam's state is in q'_α.

In particular, the only way for Adam's token to reach the accepting sink state in $\mathcal{A}_{\mathtt{reach}}$ is to read α from q'_α, but then Eve's token at ι_α in the simulation game would also reach the accepting sink state. Eve's strategy is therefore winning in the simulation games in $\mathtt{reach}(\mathcal{A}_{\mathtt{strong}})$, and it follows that $\mathcal{A}_{\mathtt{strong}}$ has reach-covering. We conclude that $\mathcal{A}_{\mathtt{strong}}$ is simplified, and therefore HD. □

The following lemma then disproves the strong-rewiring conjecture.

Lemma 3.4. *Let s be a reach-deterministic state in $\mathcal{A}_{\mathtt{strong}}$. Let $\mathcal{B}$ be the automaton obtained by replacing the transition $Y \xrightarrow{y} I$ in $\mathcal{A}_{\mathtt{strong}}$ by the transition $Y \overset{y}{\Rightarrow} s$. Then,*

$$L(\mathcal{B}) \neq L(\mathcal{A}_{\mathtt{strong}}).$$

Proof. We show that there is word accepted by $\mathcal{B}$ that is rejected by $\mathcal{A}_{\mathtt{strong}}$. We have the following three cases for the state s:

$s \in Y$. Then y^ω is accepted by $\mathcal{B}$, but is not in $L_{\mathtt{strong}}$.
$s = p_\alpha$ **or** $s = p'_\alpha$. Then $\mathcal{B}$ accepts the word $(yx\beta x\beta y)^\omega$ that is not in $L_{\mathtt{strong}}$, where $\beta \neq \alpha$ is in $\{a, b, c\}$.
$s = q_\alpha$ **or** q'_α. Then, $(x\alpha y)^\omega$ is accepted by $\mathcal{B}$ but is not in $L_{\mathtt{strong}}$ □.

Lemmas 3.3 and 3.4 together imply that the strong-rewiring conjecture is false. We next proceed towards disproving the weak-rewiring conjecture.

First, we observe that our counterexample for the strong-rewiring conjecture is not a counterexample for the weak-rewiring conjecture. We present a language-equivalent rewiring $\mathcal{D}_{\mathtt{strong}}$ in Fig. 5.

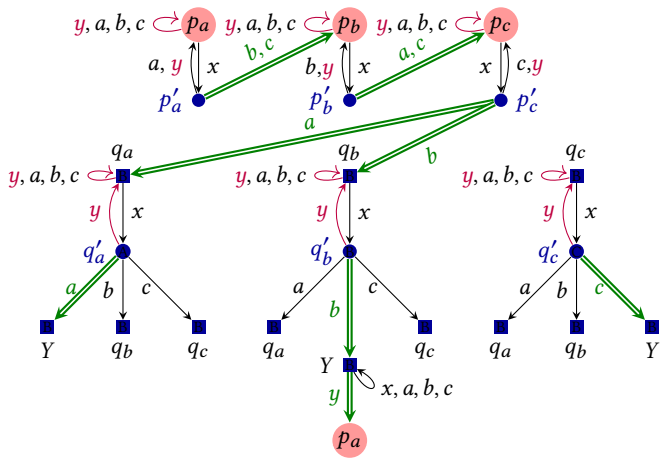


Figure 5: A language-equivalent rewiring $\mathcal{D}_{\mathtt{strong}}$ for $\mathcal{A}_{\mathtt{strong}}$.

Lemma 3.5. *The automaton $\mathcal{D}_{\mathtt{strong}}$ is a language-equivalent rewiring of $\mathcal{A}_{\mathtt{strong}}$.*

The proof of Lemma 3.5 is in the Appendix.

3.1.2 Weak rewiring ☠. We will now describe a counterexample to the weak-rewiring conjecture, which is the automaton $\mathcal{A}_{\mathtt{weak}}$ shown in Fig. 6. The automaton $\mathcal{A}_{\mathtt{weak}}$ is obtained by modifying $\mathcal{A}_{\mathtt{strong}}$ as follows:

(1) The significant transitions from p's to qs are made non-significant.
(2) For these transition to remain non-significant when the automaton is made normal, we add three new *reset letters* $\mathtt{r}_a$, $\mathtt{r}_b$, and $\mathtt{r}_c$. The transitions in $\mathcal{A}_{\mathtt{weak}}$ over these reset letters look like the following:
   (a) From every state s in $\{\iota, \iota_a, \iota_b, \iota_c\}$ we have a significant transition $s \overset{\mathtt{r}_\alpha}{\Longrightarrow} p_\alpha$ for each $\alpha \in \{a, b, c\}$.
   (b) From state Y, we have non-significant self-loops on the reset letters.
   (c) From every other state s, we have a non-significant transition $s \overset{\mathtt{r}_\alpha}{\longrightarrow} p_i$, for each $\alpha \in \{a, b, c\}$.

Changing transitions from the states p's to qs to be non-significant and adding the rest letters to obtain $\mathcal{A}_{\mathtt{weak}}$ from $\mathcal{A}_{\mathtt{strong}}$ has the effect of 'gluing' the reach-deterministic states apart from Y in a single strongly connected component in the automaton $\mathtt{reach}(\mathcal{A}_{\mathtt{weak}})$ (blue box in Fig. 6). This eliminates the possibility of a language-equivalent rewiring similar to $\mathcal{D}_{\mathtt{strong}}$ in Fig. 5, as we argue next.

First, we describe the language of $\mathcal{A}_{\mathtt{weak}}$. Let $\Sigma' = \{x, a, b, c, y, \mathtt{r}_a, \mathtt{r}_b, \mathtt{r}_c\}$ and

$$L'_{\alpha\beta\beta} = (L_\alpha + \mathtt{r}_\alpha)L_\beta L_\beta\,, \quad \text{for } \alpha, \beta \in \{a, b, c\},$$

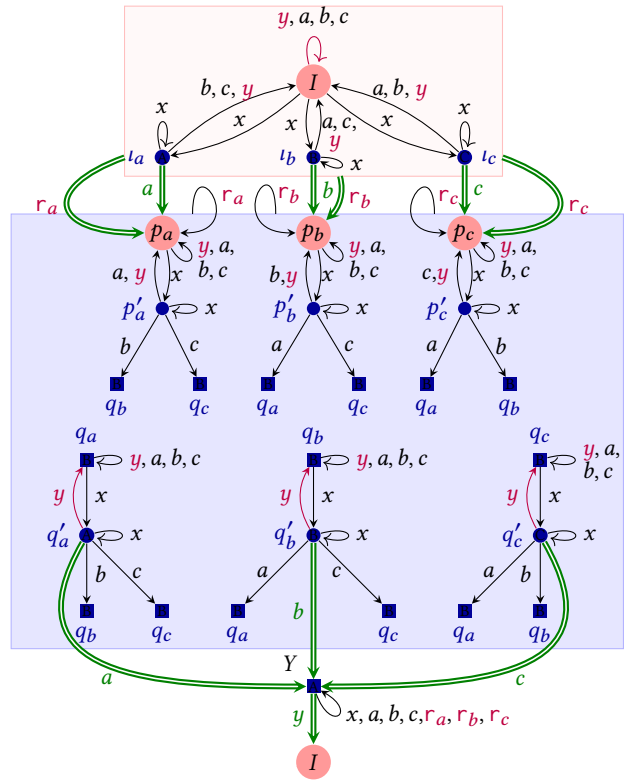


Figure 6: HD Büchi automaton $\mathcal{A}_{\mathtt{weak}}$ for which the weak rewiring conjecture is not true. The transitions $\overset{\mathtt{r}_i}{\longrightarrow} p_i$ outgoing a box indicate that from all states in the box over $\mathtt{r}_i$ we go to p_i.

where $L_\alpha = (x^*y^*)^*x\alpha$ as previously. Then,

$$L_{\mathtt{weak}} = ((x + a + b + c + y)^* \cdot \bigcup_{\alpha \neq \beta \in \{a,b,c\}} L'_{\alpha\beta\beta} \cdot \Sigma^* \cdot y)^\omega.$$

We show that $\mathcal{A}_{\mathtt{weak}}$ is a counterexample for the weak-rewiring conjecture.

Lemma 3.6. *The weak-rewiring conjecture is not true for $\mathcal{A}_{\mathtt{weak}}$.*

Proof sketch. The proof that $\mathcal{A}_{\mathtt{weak}}$ is simplified and recognises $L_{\mathtt{weak}}$ is similar to that for $\mathcal{A}_{\mathtt{strong}}$. We next show that there is no language-equivalent rewiring for $\mathcal{A}_{\mathtt{weak}}$. Let $\mathcal{B}$ be a rewiring of $\mathcal{A}_{\mathtt{weak}}$.

First, we observe that if $\mathcal{B}$ has a significant transition of the form $q'_\alpha \overset{\alpha}{\Rightarrow} s$ for some $\alpha \in \{a, b, c\}, s \neq Y$, then $\mathcal{B}$ accepts the word $(\mathtt{r}_a x b x \alpha x \alpha)^\omega$, which is not in $L_{\mathtt{weak}}$ because w does not contain infinitely many y. Thus, in this case, $\mathcal{B}$ is not language-equivalent to $\mathcal{A}_{\mathtt{weak}}$. We therefore assume that all significant transitions outgoing from states q'_a, q'_b, and q'_c are towards Y.

Suppose $\mathcal{B}$ has the transition $Y \overset{y}{\Rightarrow} s$ for some state s in $\mathcal{B}$. The proof that then $\mathcal{B}$ accepts a word that is not recognised by $\mathcal{A}_{\mathtt{strong}}$ is verbatim the proof of Lemma 3.4. This disproves the weak-rewiring conjecture. □

3.2 Copying reach-deterministic states

The previous automaton $\mathcal{A}_{\mathtt{weak}}$ may momentarily fill an optimistic researcher with the hope of having found an HD Büchi automaton strictly smaller than any equivalent deterministic one. Unfortunately, there are smaller deterministic automata recognising $L_{\mathtt{weak}}$.

In Fig. 7, we present a deterministic automaton $\mathcal{D}_{\mathtt{weak}}$ recognising $L_{\mathtt{weak}}$. This automaton is obtained by removing the non-reach-deterministic states $\{I, \iota_a, \iota_b, \iota_c\}$ from $\mathcal{A}_{\mathtt{weak}}$ and adding two copies of the state Y, to have Y_a, Y_b, and Y_c.

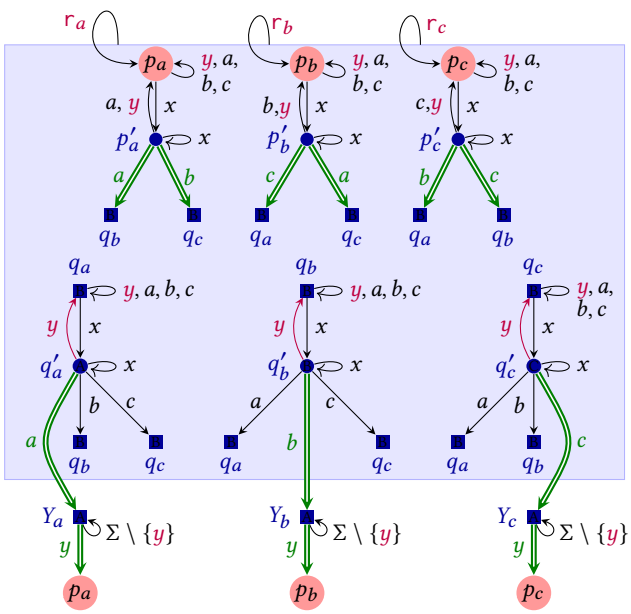


Figure 7: Determinisation of $\mathcal{A}_{\mathtt{weak}}$ by introducing copies of the state Y.

This determinisation strategy works here because the number of Y-copies (2 in this case) is smaller than the number of non-reach-deterministic states (4 in this case). To defeat it, it suffices to add many copies of the gadgets, so that we would require to introduce > 4 copies of the state Y.

3.3 Determinising the nondeterministic component

We discuss a final determinisation technique. Recall the automaton $\mathcal{A}_{\mathtt{BKKS}}$ in Fig. 1. In Fig. 3b, we showed a language-equivalent deterministic Büchi automaton of the same size that is not obtained by a rewiring, but instead by '*determinising the nondeterministic component*'. It is easy to come up with a similar language-equivalent deterministic Büchi automata for both $\mathcal{A}_{\mathtt{weak}}$ and $\mathcal{A}_{\mathtt{strong}}$, where we replace the states $I, \iota_a, \iota_b, \iota_c$ that are not reach-deterministic with two states as in Fig. 3b, and add appropriate transitions. This provides yet a different proof of non-succinctness for $\mathcal{A}_{\mathtt{weak}}$.

To defeat such a determinisation, we propose the automaton $\mathcal{A}_{\mathtt{replace}}$ in Fig. 8. This example is inspired by a recent example

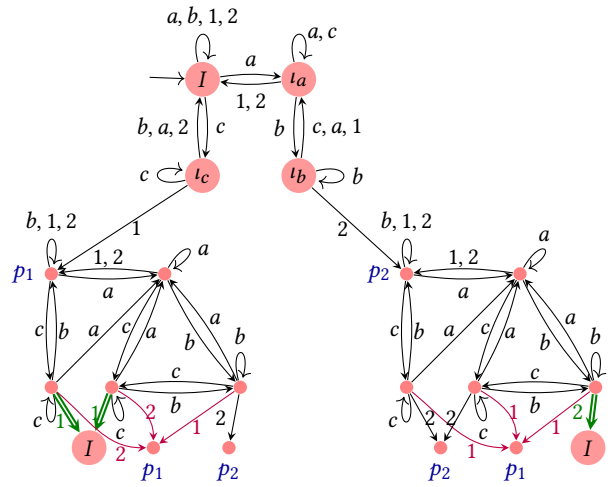


Figure 8: A HD Büchi automaton $\mathcal{A}_{\mathtt{replace}}$.

which arose in a different context [20, Figure 6]. The automaton $\mathcal{A}_{\mathtt{replace}}$ recognises the language

$$((L_1 + L_2)^*(L_1L_1 + L_2L_2))^\omega,$$

where $L_1 = \Sigma^* c1$ and $L_2 = \Sigma^* a \Sigma_A^* b2$, where $\Sigma = \{a, b, c, 1, 2\}$ and $\Sigma_A = \{a, b, c\}$. For comparison, we remark that the language of $\mathcal{A}_{\mathtt{BKKS}}$ is $(xa + xb)^*(xaxa + xbxb)^\omega$.

The only non-deterministic choice in $\mathcal{A}_{\mathtt{replace}}$ appears on I over the letter a. On this state, we guess on whether we are going to read a word in L_1 or L_2. The fact that we can use an HD resolver that eventually makes the right choice is proved similarly to the case of $\mathcal{A}_{\mathtt{BKKS}}$ (Example 1.1).

Determinising the nondeterministic component of $\mathcal{A}_{\mathtt{replace}}$, given by the states I, ι_a, ι_b and ι_c, requires 5 states (copying one of the 5-states components below). Therefore, such a determinisation strategy yields a deterministic automaton with strictly more states than $\mathcal{A}_{\mathtt{replace}}$.

There is, of course, a language-equivalent rewiring for $\mathcal{A}_{\mathtt{replace}}$, which is similar in spirit to the language-equivalent rewiring in Fig. 3a for $\mathcal{A}_{\mathtt{BKKS}}$. This provides a smaller equivalent deterministic automaton.

We are ready to propose an HD Büchi automaton strictly smaller than any equivalent deterministic one. In the next section, we combine our examples to construct the succinct HD Büchi automaton that defeats all these determinisation techniques at the same time.

4 A succinct HD Büchi Automaton

In this section, we introduce the protagonist of our succinctness result: a 65-state Büchi automaton $\mathcal{A}_{\mathtt{main}}$, and prove that it is in fact history-deterministic. In Section 5, we will prove that there is no equivalent deterministic automaton with the same number of states.

Some intuition. To build the automaton $\mathcal{A}_{\mathtt{main}}$, we combine the ideas and gadgets explained in the previous section. In particular, a

succinct HD Büchi automaton must satisfy the following (informal) properties:

No rewiring. It does not admit a language-equivalent rewiring.
No copying. A determinisation strategy based on making copies of a Y-state requires making more copies than states in the non-deterministic part.
No replacement. The non-deterministic part does not admit a determinisation with fewer states.

We ensure the no-rewiring property by using the "skeleton structure" of the automaton $\mathcal{A}_{\mathtt{weak}}$ from Fig. 6. That is, the general structure of the automaton $\mathcal{A}_{\mathtt{main}}$ consists of

- a non-deterministic initial component with 4 states,
- several deterministic "basic blocks", each of them recognising a language of finite words L_i,
- a state Y checking for the appearences of letter y.

Moreover, we include reset letters $\mathtt{r}_i$ as in $\mathcal{A}_{\mathtt{weak}}$ to 'glue together' the basic blocks in a single SCC of $\mathtt{reach}(\mathcal{A}_{\mathtt{main}})$.

We ensure the no-copying property by having $12 = 2 \cdot 6$ "basic blocks" in the skeleton structure. A determinisation via making copies of a Y-state would incurr in adding 5 states, more than what we gain from removing the non-deterministic part.

To ensure the no-replacement property, we use the gadget from Fig. 8. Each basic block, therefore, will be a subautomaton of 5 states similar to the one at the bottom left of Fig. 8.

The language. Before introducing the automaton, we describe the language it recognises. We fix the alphabet

$$\Sigma = \{a, b, c, 1, 2, 3, 4, 5, 6, \mathtt{r}_1, \mathtt{r}_2, \mathtt{r}_3, \mathtt{r}_4, \mathtt{r}_5, \mathtt{r}_6, y\}.$$

For notational convenience we let $E = \{1, 2, 3\}$, $F = \{4, 5, 6\}$ and $A = \{a, b, c\}$.

The language $L_{\mathsf{main}} = L(\mathcal{A}_{\mathtt{main}})$ is built similarly to the language of $\mathcal{A}_{\mathtt{strong}}$. The "building blocks" of the language are 6 languages of finite words, $L_1, \dots, L_6$ described below. Then, we let:

$$L_{\alpha\beta\beta} = (L_\alpha + \mathtt{r}_\alpha)L_\beta L_\beta, \quad \text{for } \alpha \neq \beta, \quad \alpha, \beta \in \{1, \dots, 6\}.$$

Then, the final language is:

$$L_{\mathsf{main}} = (\Sigma^* \cdot \bigcup_{\substack{\alpha \neq \beta \\ 1 \leq \alpha, \beta \leq 6}} L_{\alpha\beta\beta} \cdot \Sigma^* \cdot y)^\omega$$

That is, L_{main} consists of the words containing infinitely often infixes in some $L_{\alpha\beta\beta}$ as well as containing infinitely many y's.

It remains to describe the languages L_α for $\alpha = 1, \dots, 6$. This is best done by providing the *DFA-classifier* $\mathcal{A}_{\mathtt{class}}$ from Figure 9. This is a DFA over Σ with multiple final states, in this case, the six states $\blacksquare\ell_1, \dots, \blacksquare\ell_6$. Its initial state is p. The language L_α is the set of words for which the run over $\mathcal{A}_{\mathtt{class}}$ ends up in $\blacksquare\ell_\alpha$. We note that $\blacksquare\ell_\alpha$ is *not* a sink: words that admit a proper prefix ending in some $\blacksquare\ell_\alpha$ are rejected. We also remark that $\mathcal{A}_{\mathtt{class}}$ contains no $\mathtt{r}_\alpha$ transitions: all words containing such a letter are rejected. By definition, the sets $L_1, \dots, L_6$ are pairwise disjoint. The following claim directly follows from the definition.

Claim 4.1. *Let w be a finite word containing some infix in a language L_α for $\alpha \in [6]$. Then, there is some $\alpha' \in [6]$ such that w contains either a prefix in some $L_{\alpha'}$, or in $\Sigma^* \mathtt{r}_{\alpha'}$.*

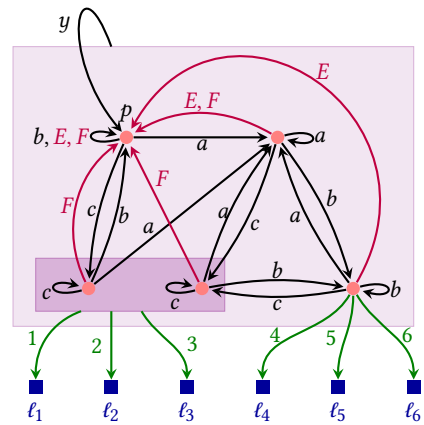


Figure 9: The DFA-classifier defining the languages $L_1, \dots, L_6$.

We give some intuition for the DFA-classifier $\mathcal{A}_{\mathtt{class}}$. First, note that y acts as a reset letter: over all main states, it goes back to state p. While no letter a is produced, the automaton stays in the two leftmost states; if the factor ce is read for some $e \in E = \{1, 2, 3\}$ is then read, it goes to some $\blacksquare\ell_e$. When letter a is read, we go to the right part of the automaton, and remain there while only letters in $A = \{a, b, c\}$ appear. From this right part:

- A^*ce make us advance to $\blacksquare\ell_e$ for $e \in E$, while A^*cF resets to p,
- A^*bf make us advance to $\blacksquare\ell_f$ for $f \in F$, while A^*bE resets to p, and
- $A^*a(E + F)$ resets to p.

The automaton $\mathcal{A}_{\mathtt{main}}$. We describe the HD Büchi automaton recognising L_{main}. The states of the automaton are:

- the initial state I, and states I_a, I_b and I_c,
- the "circular states": $\bullet p_i, \bullet q_i, \bullet r_i, \bullet s_i, \bullet t_i$ for $i \in \{1, \dots, 6\}$;
- the "square states": $\blacksquare p_i, \blacksquare q_i, \blacksquare r_i, \blacksquare s_i, \blacksquare t_i$ for $i \in \{1, \dots, 6\}$;
- and finally the state $\blacksquare Y$.

Visualising the automaton. The transitions are described in Figures 10a and 10b. To visualise the 65 state automaton without creating a tangle of spaghetti, we split the transitions across two figures and use a few graphical shorthands similar to the earlier section. The transitions of the automaton over the letters a, b, c, and y are described in Fig. 10a, whereas the transitions of the automaton over the rest of the letters $1, \dots, 6$ and $\mathtt{r}_1, \dots, \mathtt{r}_6$ are depicted in Fig. 10b. Note that to maintain the planarity and readability of the diagrams, some states appear in multiple locations. We use variables to represent sets of transitions:

- The variable i ranges over the set $\{1, \dots, 6\}$,
- the variable e ranges over the set $E = \{1, 2, 3\}$, and
- the variable f ranges over the set $F = \{4, 5, 6\}$.

Therefore, a single arrow to a state like $\bullet p_f$ in fact represents the transitions $\to \bullet p_4$, $\to \bullet p_5$, and $\to \bullet p_6$. Finally, the rectangles around states represent grouped sources. An arrow originating from a box around a set of states implies a transition from every

state within that box. For instance, in Fig. 10b, the arrow labelled y to the state $\bullet p_i$ indicates that on input y, the states $\bullet p_i, \bullet q_i, \bullet r_i, \bullet s_i$, and $\bullet t_i$ transition to state $\bullet p_i$.

Working of the automaton. We note that there is a single non-deterministic choice: when reading letter a from state I (similar to the mechanism in Figure 8).

This automaton works as follows. It reads sequences of languages L_i until a factor $L_\beta L_\beta$ that has been preceded by some L_α or r_α, with $\alpha \neq \beta$, appears. Then, it goes to state Y and waits for letter y to appear. When this happens, it restarts again.

The top box (states I, I_a, I_b, I_c) waits for the first language L_α to appear (or letter r_α). Using the non-deterministic choice on I over a, we will take a guess on whether $\alpha \in E$ and we are going to see a word of the form A^*cE (we should wait in I), or $\alpha \in F$ and we are going to see a word of the form A^*aA^*bF.

If we make the right choice, we move to $\bullet p_\alpha$. If we make the wrong choice, no big deal! We go back to I, and since for words in the language, we know that some factor $L_\beta L_\beta$ must appear, we can use the information "$\alpha \in E$?" to resolve the non-determinism the next time: if $\alpha \in E$, we wait in I (loop at state I on letter a), if $\alpha \in F$, we go to I_a whenever possible. Eventually, this strategy will make us advance to the big blue box over a word in L_{main}.

In the big blue box, the αth box on the top row resets over L_α, and moves to the lower row over L_β if $\beta \neq \alpha$. When we say reset or move to, we mean that it specifically $\bullet p_\alpha$ and to the state $\blacksquare p_\beta$, respectively. From $\blacksquare p_\beta$ in the lower row, it advances to Y over L_β. Over L_α, for $\alpha \neq \beta$, it stays on the lower row, going to $\blacksquare p_\alpha$. Over a letter r_i we reset to the upper row, to $\bullet i$; this ensures that the full big blue box is strongly connected.

The automaton has three strongly connected components when restricted to non-significant transitions: the top pink box, the big blue box, and the state Y.

Correctness. The proof of correctness of $\mathcal{A}_{\mathsf{main}}$ is similar to the proof of Lemma 3.3. Full proofs are provided in Appendix B.

Theorem 4.2. *The automaton $\mathcal{A}_{\mathsf{main}}$ is history deterministic and recognises L_{main}. Moreover, it is simplified.*

5 There is no Small Deterministic Automaton

In this Section, we prove that $\mathcal{A}_{\mathsf{main}}$ is succinct.

Theorem 5.1. *Every deterministic language-equivalent Büchi automaton of $\mathcal{A}_{\mathsf{main}}$ has at least 66 states.*

To prove Theorem 5.1 above, we prove the equivalent result that every deterministic coBüchi automaton recognising L^c_{main} has at least 66 states. Towards this, we rely on results from Abu Radi and Kupferman on the minimisation of HD coBüchi automata [2], which we describe in Section 5.1. We then present computer-aided lemmas, which concern establishing the size of minimal DFAs separating regular languages over finite words, an NP-complete problem [12]. Using them, we then conclude our proof in Section 5.3.

5.1 Structure of HD coBüchi automata

We briefly recall the results of Abu Radi and Kupferman's seminal paper [2] that showed a canonical form for HD coBüchi automata, which we will require. We will then use their results to construct a state-minimal history-deterministic coBüchi automaton for the complement language. We begin by introducing a few notions that are necessary to state their results.

We say that a coBüchi automaton is a *safety automaton* if it has no significant transitions. For every coBüchi automaton $\mathcal{A}$, we use $\mathsf{safe}(\mathcal{A})$ to denote the safety automaton that is obtained by deleting all the significant transitions of $\mathcal{A}$.

A *safe component* of $\mathcal{A}$ is a strongly connected component of $\mathsf{safe}(\mathcal{A})$. We let $\mathsf{Safe}(\mathcal{A})$ denote the set of safe-components of $\mathcal{A}$. We write $L_{\mathsf{safe}}(\mathcal{A}, q)$ to denote the language $L(\mathsf{safe}(\mathcal{A}), q)$, for a state q in $\mathcal{A}$, which we call the *safe language* of q.

We say that a coBüchi automaton is *normal* if every transition that is not in any safe component of $\mathcal{A}$ is a significant transition. Every coBüchi automaton can be made normal while preserving the acceptance of each run, by simply changing every non-significant transitions of $\mathcal{A}$ that is outside a safe-component of $\mathcal{A}$ to a significant transition. We assume that all states of automata are reachable from the initial state.

A coBüchi automaton $\mathcal{A}$ is *safe-deterministic* if the automaton $\mathsf{safe}(\mathcal{A})$ is deterministic. A coBüchi automaton $\mathcal{A}$ is *safe-minimal* if there are no two states p, q such that p and q are language-equivalent in $\mathcal{A}$ as well as in $\mathsf{safe}(\mathcal{A})$. We write that a coBüchi automaton $\mathcal{A}$ is *safe-centralised* if there are no two states p, q in different safe-components of $\mathcal{A}$ such that p and q are language-equivalent in $\mathcal{A}$ and $L_{\mathsf{safe}}(\mathcal{A}, p) = L_{\mathsf{safe}}(\mathcal{A}, q)$.

Abu Radi and Kupferman proved the following result on minimality for HD coBüchi automata.

Lemma 5.2 ([2, Theorem 3.6])**.** *Let $\mathcal{A}$ be a history-deterministic coBüchi automaton that is normal, semantically-deterministic, safe-deterministic, safe-minimal, and safe-centralised. Then, it is a state-wise minimal amongst all HD coBüchi automaton recognising $L(\mathcal{A})$.*

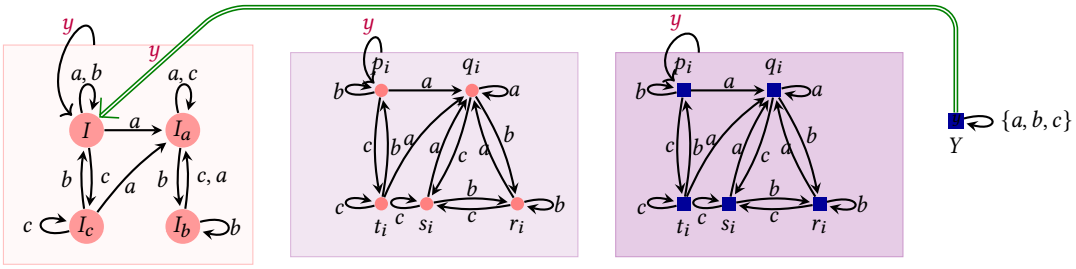
We also use the subsequent result from Abu Radi and Kupferman's work.

Lemma 5.3 ([2, Proposition 3.4])**.** *Let $\mathcal{A}$ and $\mathcal{B}$ be two language-equivalent history-deterministic coBüchi automata that are normal and safe-deterministic. Then for every state p in $\mathcal{A}$, there are states q in $\mathcal{A}$ and s in $\mathcal{B}$ such that $L(\mathcal{A}, p) = L(\mathcal{A}, q) = L(\mathcal{B}, s)$ and*

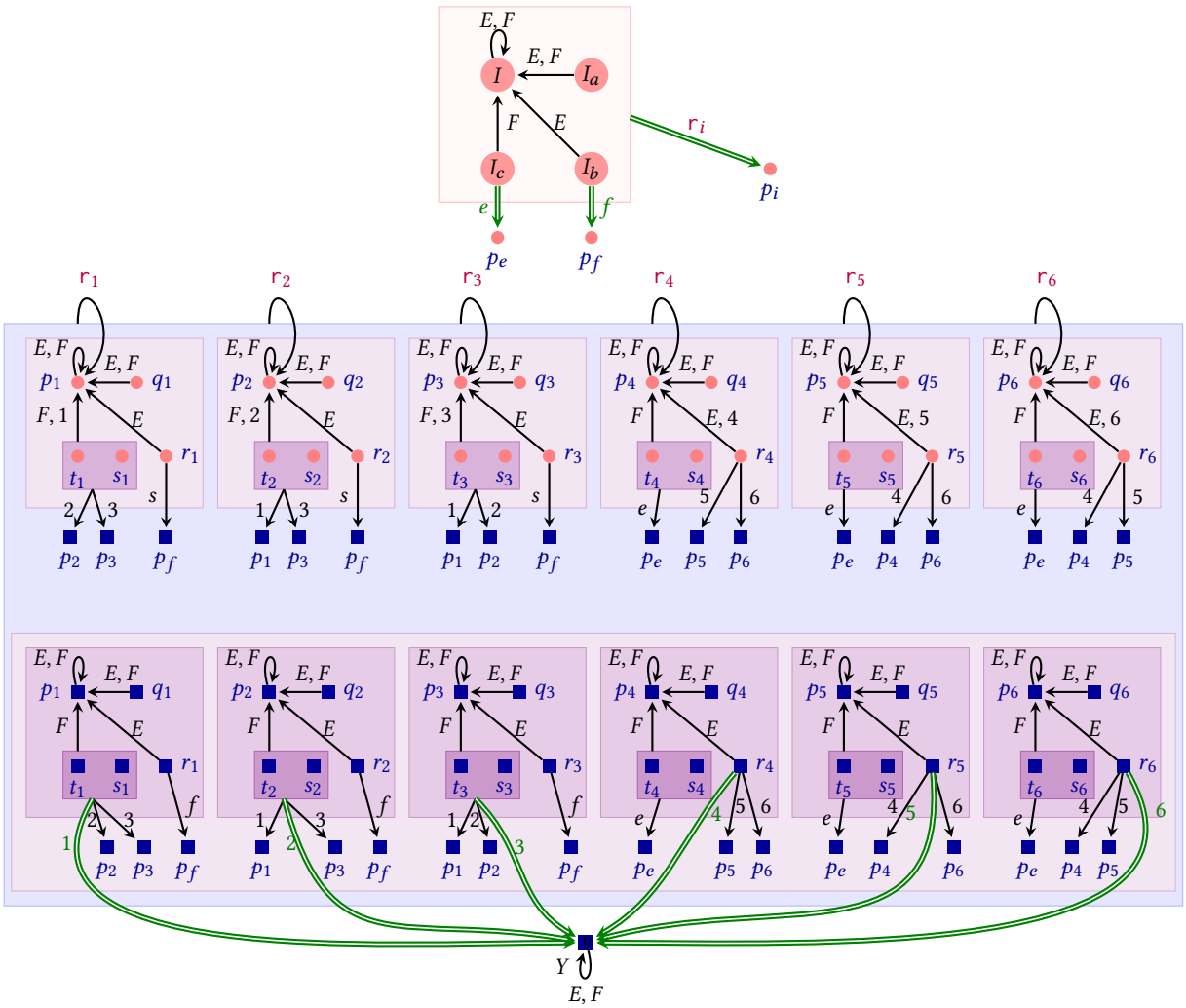
$$L_{\mathsf{safe}}(\mathcal{A}, p) \subseteq L_{\mathsf{safe}}(\mathcal{A}, q) = L_{\mathsf{safe}}(\mathcal{B}, s).$$

5.2 Complementation of HD Büchi automata

In their 2025 paper, Abu Radi, Kupferman, and Leshkowitz describe a complementation procedure for HD Büchi automata that produces HD coBüchi automata with at most as many states [28, Theorem 2]. This procedure builds an HD coBüchi automata for the complement language that has as set of states the reach-deterministic states of a simplified HD Büchi automata. If we use their procedure on our 65-state automaton $\mathcal{A}_{\mathsf{main}}$, we obtain a 61-state history-deterministic coBüchi automaton C_{main}. The automaton C_{main} has its states as the reach-deterministic states of $\mathcal{A}_{\mathsf{main}}$. The non-significant transitions of C_{main} are the outgoing non-significant transitions from the reach-deterministic states of $\mathcal{A}_{\mathsf{main}}$. The significant transitions of C_{main} are transitions $p \overset{a}{\Rightarrow} q$ for every pair of states p, q in C_{main} and every letter a. A rendering of the automaton is given in Fig. 13 in the Appendix. It follows from Abu Radi, Kupferman, and Leshkowitz's



(a) The automaton's transitions over the letters a, b, c, and y. The value of i ranges over the set $\{1, \ldots, 6\}$.



(b) The transitions over the alphabet $\{1, \ldots, 6\} \cup \{\mathsf{r}_1, \ldots, \mathsf{r}_6\}$. We let $E = \{1, 2, 3\}$ and $F = \{4, 5, 6\}$. We also assume that e ranges over the set E and f over the set F.

Figure 10: The automaton $\mathcal{A}_{\mathsf{main}}$ whose transitions are depicted in two parts

work [28, Proof of Theorem 9] that $L(C_{\mathsf{main}}) = L^c_{\mathsf{main}}$; we include a proof in the appendix for self-containment nevertheless.

Lemma 5.4. *The automata C_{main} is history-deterministic, and recognises the complement of the language recognised by $\mathcal{A}_{\mathsf{main}}$.*

We proof of statewise-minimality for C_{main}, stated next, is included in Appendix C.

Lemma 5.5. *The automaton C_{main} is the statewise smallest HD coBüchi automaton recognising $L(C_{\mathsf{main}})$.*

Proof sketch. From Lemma 5.2, we need to prove that $\mathcal{C}_{\mathtt{main}}$ is HD, semantically-deterministic, safe-deterministic, normal, safe-minimal, and safe-centralised. By Lemma 5.4, $\mathcal{C}_{\mathtt{main}}$ is HD. It is also SD because every pair of states has a transition from one to the other on every letter, and $\mathcal{C}_{\mathtt{main}}$ is safe-deterministic and normal by construction. Safe-centralisation follows easily, as they are only 2 safe-components. To show that $\mathcal{C}_{\mathtt{main}}$ is safe-minimised, a case analysis is needed. □

We note that Lemma 5.5 also proves that every deterministic coBüchi automaton $\mathcal{D}$ recognising $L(\mathcal{C}_{\mathtt{main}})$ requires at least 61 states. We thus need to prove that any such $\mathcal{D}$ would require at least 5 more states, which we prove in Section 5.3.

5.3 Proof of succinctness

We show the following result.

Lemma 5.6. *Every deterministic coBüchi automaton that is language-equivalent to $\mathcal{C}_{\mathtt{main}}$ has at least 66 states.*

Throughout this subsection, we fix $\mathcal{D}_{\mathtt{main}}$ to be a deterministic coBüchi automaton that recognises $L(\mathcal{C}_{\mathtt{main}})$. We assume, without loss of generality, that $\mathcal{D}_{\mathtt{main}}$ is normal, that is, all its non-significant transitions occur in some SCC in $\mathtt{safe}(\mathcal{D}_{\mathtt{main}})$. Note that because $L(\mathcal{C}_{\mathtt{main}}) = L(\mathcal{D}_{\mathtt{main}})$ is prefix-independent, every state of $\mathcal{D}_{\mathtt{main}}$ recognises the language $L(\mathcal{D}_{\mathtt{main}})$.

We denote the safe-component of $\mathcal{C}_{\mathtt{main}}$ which consists of every state of $\mathcal{C}_{\mathtt{main}}$ apart from Y by $\mathcal{S}^{\mathcal{C}}_{\mathtt{safe}}$. We first prove existence of states in $\mathcal{D}_{\mathtt{main}}$ whose safe-languages coincide with the states of $\mathcal{S}^{\mathcal{C}}_{\mathtt{safe}}$ in $\mathcal{C}_{\mathtt{main}}$ (Corollary 5.8). For this, we use Lemma 5.3.

Lemma 5.7. *There is a state d_0 in $\mathcal{D}_{\mathtt{main}}$ such that*

$$L_{\mathtt{safe}}(\mathcal{D}_{\mathtt{main}}, d_0) = L_{\mathtt{safe}}(\mathcal{C}_{\mathtt{main}}, \bullet p_1).$$

Proof. In Lemma 5.3, we let $\mathcal{C}_{\mathtt{main}}$ be $\mathcal{A}$, $\mathcal{D}_{\mathtt{main}}$ be $\mathcal{B}$ and let p be $\bullet p_1$. Then, there is a state q in $\mathcal{C}_{\mathtt{main}}$ and a state r in $\mathcal{D}_{\mathtt{main}}$ such that

$$L_{\mathtt{safe}}(\mathcal{C}_{\mathtt{main}}, \bullet p_1) \subseteq L_{\mathtt{safe}}(\mathcal{C}_{\mathtt{main}}, q) = L_{\mathtt{safe}}(\mathcal{D}_{\mathtt{main}}, r).$$

Because $\mathcal{C}_{\mathtt{main}}$ is safe-centralised (see proof of Lemma 5.5), we know that $\bullet p_1$ and q are in the same safe-component $\mathcal{S}^{\mathcal{C}}_{\mathtt{safe}}$ of $\mathcal{C}_{\mathtt{main}}$. Thus, let d_0 be such that we have the non-significant transition $r \xrightarrow{\mathtt{r}_1} d_0$ in $\mathcal{D}_{\mathtt{main}}$, which we know exists because $L_{\mathtt{safe}}(\mathcal{C}_{\mathtt{main}}, q) = L_{\mathtt{safe}}(\mathcal{D}_{\mathtt{main}}, r)$ and $q \xrightarrow{\mathtt{r}_1} \bullet p_1$ is a non-significant transition in $\mathcal{C}_{\mathtt{main}}$. It follows that the safe-language of d_0 in $\mathcal{D}_{\mathtt{main}}$ is $L_{\mathtt{safe}}(\mathcal{C}_{\mathtt{main}}, \bullet p_1)$, as desired. □

We let $\mathcal{S}^{\mathcal{D}}_{\mathtt{safe}}$ be the safe-component of $\mathcal{D}_{\mathtt{main}}$ that contains d_0. Since $\mathcal{C}_{\mathtt{main}}$ is safe-minimal, we easily obtain the following corollary.

Corollary 5.8. *For every state p in $\mathcal{S}^{\mathcal{C}}_{\mathtt{safe}}$, there is a state q in $\mathcal{S}^{\mathcal{D}}_{\mathtt{safe}}$ such that $L_{\mathtt{safe}}(\mathcal{D}_{\mathtt{main}}, q) = L_{\mathtt{safe}}(\mathcal{C}_{\mathtt{main}}, p)$.*

In particular, $\mathcal{S}^{\mathcal{D}}_{\mathtt{safe}}$ has at least 60 states. Furthermore, Similar to Lemma 5.7, we prove that there is a state d_y in $\mathcal{D}_{\mathtt{main}}$ whose safe-language is equivalent to that of Y in $\mathcal{A}_{\mathtt{main}}$.

Lemma 5.9. *There is a state d_y in $\mathcal{D}_{\mathtt{main}}$ such that*

$$L_{\mathtt{safe}}(\mathcal{D}_{\mathtt{main}}, d_y) = L_{\mathtt{safe}}(\mathcal{C}_{\mathtt{main}}, Y).$$

Proof. We prove that there is a state d_y in $\mathcal{D}_{\mathtt{main}}$ and a state q in $\mathcal{C}_{\mathtt{main}}$ such that

$$L_{\mathtt{safe}}(\mathcal{C}_{\mathtt{main}}, Y) \subseteq L_{\mathtt{safe}}(\mathcal{C}_{\mathtt{main}}, q) = L_{\mathtt{safe}}(\mathcal{D}_{\mathtt{main}}, d_y).$$

We observe that q must be Y, since $\mathcal{C}_{\mathtt{main}}$ is safe-centralised and the safe-component consisting of Y only consists of Y. Thus, the above language inclusion is an equality, as desired. □

Thus $\mathcal{D}_{\mathtt{main}}$ has at least 61 states: at least 60 states in $\mathcal{S}^{\mathcal{D}}_{\mathtt{safe}}$, and one state d_y, and possibly more. Towards proving that $\mathcal{D}_{\mathtt{main}}$ has at least 66 states, we need to introduce a few notions.

The subautomaton $\mathcal{D}_{\mathtt{core}}$. First, we consider the safety automaton $\mathcal{S}^{\mathcal{D}}_{\mathtt{safe}}$ without the transitions on y, and we consider the SCC decomposition of $\mathcal{S}^{\mathcal{D}}_{\mathtt{safe}}$ in the absence of transitions on y. We call this the y-SCC decomposition. We use $y\mathsf{Safe}(\mathcal{D}_{\mathtt{main}})$ to denote the set of the resulting strongly-connected components. For a state p in $\mathcal{S}^{\mathcal{D}}_{\mathtt{safe}}$, we use $L_{y\mathtt{safe}}(p)$ to denote the language of words recognised from p in a SCC of the y-SCC decomposition of $\mathcal{S}^{\mathcal{D}}_{\mathtt{safe}}$. That is, a word $w \in L_{y\mathtt{safe}}(p)$ if and only if the run of p on w contains no transitions on y, and stays in the same SCC in the y-SCC decomposition of $\mathcal{S}^{\mathcal{D}}_{\mathtt{safe}}$.

For the y-SCC-decomposition of $\mathcal{S}^{\mathcal{D}}_{\mathtt{safe}}$, let $\mathcal{D}_{\mathtt{core}}$ be a subautomaton that is an end-SCC. We claim that $\mathcal{D}_{\mathtt{core}}$ has at least 60 states.

Lemma 5.10. *For every state q in $\mathcal{S}^{\mathcal{C}}_{\mathtt{safe}}$, the subautomaton $\mathcal{D}_{\mathtt{core}}$ contains a state p such that $L_{\mathtt{safe}}(\mathcal{D}_{\mathtt{main}}, p) = L_{\mathtt{safe}}(\mathcal{C}_{\mathtt{main}}, q)$.*

Proof. Let p' be a state in $\mathcal{D}_{\mathtt{core}}$. Then, by construction, we know there is a state q' in $\mathcal{S}^{\mathcal{C}}_{\mathtt{safe}}$ such that

$$L_{\mathtt{safe}}(\mathcal{D}_{\mathtt{main}}, p') = L_{\mathtt{safe}}(\mathcal{C}_{\mathtt{main}}, q').$$

The result then follows from the observation that $\mathcal{S}^{\mathcal{C}}_{\mathtt{safe}}$ is strongly-connected even in the absence of transitions on y. Indeed, for each state p in $\mathcal{S}^{\mathcal{C}}_{\mathtt{safe}}$, there is a word $u \in (\Sigma \setminus \{y\})^*$ such that $p' \xrightarrow{u} p \in \mathcal{S}^{\mathcal{C}}_{\mathtt{safe}}$, and the corresponding run from q' on u in $\mathcal{D}_{\mathtt{main}}$ leads to a state q that is in $\mathcal{D}_{\mathtt{core}}$ because $\mathcal{D}_{\mathtt{core}}$ is an end-component. Then $L_{\mathtt{safe}}(\mathcal{D}_{\mathtt{main}}, p) = L_{\mathtt{safe}}(\mathcal{C}_{\mathtt{main}}, q)$, as desired. □

Thus, we note that if the y-SCC decomposition of $\mathcal{S}^{\mathcal{D}}_{\mathtt{safe}}$ contains two end components, then the automaton $\mathcal{D}_{\mathtt{main}}$ has size at least 120. We thus suppose that the y-SCC decomposition of $\mathcal{S}^{\mathcal{D}}_{\mathtt{safe}}$ has exactly one end component, which we call $\mathcal{D}_{\mathtt{core}}$.

Assumption 5.11. *The y-SCC decomposition of $\mathcal{S}^{\mathcal{D}}_{\mathtt{safe}}$ has exactly one end component, which we denote by $\mathcal{D}_{\mathtt{core}}$.*

$\mathcal{K}$-safety. Let $\mathcal{K}$ be the language that recognises the set of infinite words for which no prefix is in L_i for $i \in [6]$. This language $\mathcal{K}$ is recognised by the safety automaton shown in Fig. 11. We let $\mathcal{K}_y$ be the language that recognises the subset of words in $\mathcal{K}$ that contain no y.

We now state the lemma for which we provide a computer-assisted proof.

Lemma 5.12. *Let $\mathcal{D}$ be a deterministic safety automaton that separates $\mathcal{K}$ (resp. $\mathcal{K}_y$) and $L_{\mathtt{safe}}(\mathcal{C}_{\mathtt{main}}, q)$ for some state q in $\mathcal{S}^{\mathcal{C}}_{\mathtt{safe}}$, i.e.,*

$$\mathcal{K} \subseteq L(\mathcal{D}) \subseteq L_{\mathtt{safe}}(\mathcal{C}_{\mathtt{main}}, q).$$

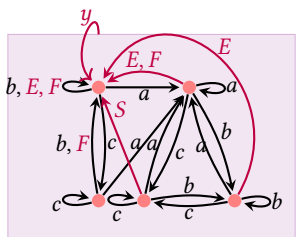


Figure 11: The automaton for language $\mathcal{K}$. The initial state it the top left one. The automaton for language $\mathcal{K}_y$ is obtained by removing the transitions on y.

Then, $\mathcal{D}$ has at least 5 states.

We note that separating regular languages given by DFAs (or deterministic safety automata) is an NP-complete problem [12]. We thus use DFAMiner, a SAT-solver based tool for separating regular languages [10]. Instead of plugging a 60-state and a 5-state automaton into this tool, however, we make certain reductions that allows us to considerably reduce the state-space of automata that we input. We describe precise details in Appendix D. We note two useful corollaries of Lemma 5.12 below.

Corollary 5.13. *Let q be a state of $\mathcal{D}_{\mathtt{main}}$ outside $\mathcal{S}^{\mathcal{D}}_{\mathtt{safe}}$. If*

$$\mathcal{K} \subseteq L_{\mathtt{safe}}(\mathcal{D}_{\mathtt{main}}, q),$$

then the safe-component containing q has at least 5 states.

Proof. Due to Lemma 5.3, we know that there is a state p in $C_{\mathtt{main}}$ such that $L_{\mathtt{safe}}(\mathcal{D}_{\mathtt{main}}, q) \subseteq L_{\mathtt{safe}}(C_{\mathtt{main}}, p)$. This state p can't be Y, since $\mathcal{K} \nsubseteq L_{\mathtt{safe}}(C_{\mathtt{main}}, Y)$. Thus, p is a state in $\mathcal{S}^{C}_{\mathtt{safe}}$. The conclusion then follows from Lemma 5.12. □

Corollary 5.14. *Let q be a state of $\mathcal{S}^{\mathcal{D}}_{\mathtt{safe}}$ outside $\mathcal{D}_{\mathtt{core}}$. If*

$$\mathcal{K}_y \subseteq L_{y\mathtt{safe}}(q),$$

then the y-SCC containing q has at least 5 states.

Proof. Observe that $L_{y\mathtt{safe}}(q) \subseteq L_{\mathtt{safe}}(\mathcal{D}_{\mathtt{main}}, q)$. Since q is in $\mathcal{S}^{\mathcal{D}}_{\mathtt{safe}}$, there is a state $p \in \mathcal{S}^{C}_{\mathtt{safe}}$ such that

$$L_{\mathtt{safe}}(\mathcal{D}_{\mathtt{main}}, q) = L_{\mathtt{safe}}(C_{\mathtt{main}}, p).$$

We thus obtain that $\mathcal{K}_y \subseteq L_{y\mathtt{safe}}(q) \subseteq L_{\mathtt{safe}}(C_{\mathtt{main}}, p)$, and the conclusion follows from Lemma 5.12. □

Observe that if there is a state q outside $\mathcal{S}^{\mathcal{D}}_{\mathtt{safe}}$ with

$$\mathcal{K} \subseteq L_{\mathtt{safe}}(\mathcal{D}_{\mathtt{main}}, q),$$

then the safe-component of q is disjoint from the safe-component consisting of d_y, since $\mathcal{K} \nsubseteq L_{\mathtt{safe}}(\mathcal{D}_{\mathtt{main}}, d_y)$ and $L_{\mathtt{safe}}(\mathcal{D}_{\mathtt{main}}, d_y)$ is prefix-independent. Thus, in this case, $\mathcal{D}_{\mathtt{main}}$ will have at least 66 states: the 60 states of $\mathcal{D}_{\mathtt{core}}$, the state d_Y, and these additional 5 states that Corollary 5.13 implies the existence of.

Similarly, if there is a state q outside $\mathcal{D}_{\mathtt{core}}$ with $\mathcal{K}_y \subseteq L_{y\mathtt{safe}}(q)$ then Corollary 5.14 implies that $\mathcal{D}_{\mathtt{main}}$ has at least 66 states: the 60 states of $\mathcal{D}_{\mathtt{core}}$, the state d_Y, and these additional 5 states that Corollary 5.14 implies the existence of.

We thus assume the following statement.

Assumption 5.15. *In $\mathcal{D}_{\mathtt{main}}$, there are no states q outside $\mathcal{S}^{\mathcal{D}}_{\mathtt{safe}}$ with $\mathcal{K} \subseteq L_{\mathtt{safe}}(\mathcal{D}_{\mathtt{main}}, q)$ and there are no states p in $\mathcal{S}^{\mathcal{D}}_{\mathtt{safe}}$ outside $\mathcal{D}_{\mathtt{core}}$ with $\mathcal{K}_y \subseteq L_{y\mathtt{safe}}(p)$.*

The next lemma concludes the proof.

Lemma 5.16. *Under Assumption 5.15, there are at least 6 states outside $\mathcal{D}_{\mathtt{core}}$.*

Lemma 5.16 above implies that there are at least six distinct state $\pi_1, \pi_2, \ldots, \pi_6$ apart from at least 60 states in $\mathcal{D}_{\mathtt{core}}$. Thus, $\mathcal{D}_{\mathtt{main}}$ has at least 66 states. This finishes the proof of Lemma 5.6, and hence Theorem 1.2.

6 Conclusion and future directions

In this paper, we have solved the succinctness question for history-deterministic Büchi automata by exhibiting a 65-state HD Büchi automaton such that any language-equivalent deterministic Büchi automaton requires at least 66 states. The construction and the proof are highly involved and required computer assistance for some intermediate steps. In order to have a better understanding of these automata and make progress in further questions, a first step would be to find a simpler example of a succinct history-deterministic Büchi automaton. The existence of such automata is immediately unclear, even the smallest Büchi automaton that was know is not determinisable by pruning has 7 states.

The immediate next question is to settle the width of the gap between deterministic and HD Büchi automata. More precisely: is there a family of HD Büchi automata such that equivalent deterministic Büchi automata need at least a quadratic number of states? While our result effectively kills the "no gap" conjecture (and rules out approaches like Conjecture 3.1), constructing a family with a quadratic gap remains a challenge, especially given the complexity required just to establish a separation of one state.

Beyond theory, the techniques we developed to establish lower bounds on deterministic state complexity have practical potential. An interesting future direction would be to integrate these heuristics into SAT-based minimization tools like SPOT [11]. More precisely, given a deterministic Büchi automaton we can find a non-trivial lower bound on the size of any equivalent deterministic (or HD) automaton by minimising the coBüchi automaton for its complement language, following [1] and as described in Section 5.2.

References

[1] Bader Abu Radi and Orna Kupferman. 2022. Minimization and Canonization of GFG Transition-Based Automata. *Log. Methods Comput. Sci.* 18, 3 (2022). doi:10.46298/lmcs-18(3:16)2022

[2] Bader Abu Radi and Orna Kupferman. 2023. On Semantically-Deterministic Automata. In *International Colloquium on Automata, Languages, and Programming, ICALP 2023 (LIPIcs, Vol. 261)*. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 109:1–109:20. https://doi.org/10.4230/LIPIcs.ICALP.2023.109

[3] Rohan Acharya, Marcin Jurdziński, and Aditya Prakash. 2024. Lookahead Games and Efficient Determinisation of History-Deterministic Büchi Automata. In *51st International Colloquium on Automata, Languages, and Programming (ICALP 2024) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 297)*, Karl Bringmann, Martin Grohe, Gabriele Puppis, and Ola Svensson (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 124:1–124:18. doi:10.4230/LIPIcs.ICALP.2024.124

[4] Udi Boker, Denis Kuperberg, Orna Kupferman, and Michal Skrzypczak. 2013. Nondeterminism in the Presence of a Diverse or Unknown Future. In *Automata, Languages, and Programming - 40th International Colloquium, ICALP 2013, Riga, Latvia, July 8-12, 2013, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 7966)*, Fedor V. Fomin, Rusins Freivalds, Marta Z. Kwiatkowska, and David Peleg (Eds.). Springer, 89–100. doi:10.1007/978-3-642-39212-2_11

[5] Udi Boker and Karoliina Lehtinen. 2023. When a Little Nondeterminism Goes a Long Way: An Introduction to History-Determinism. *ACM SIGLOG News* 10, 1 (2023), 24–51. doi:10.1145/3584676.3584682

[6] Antonio Casares, Thomas Colcombet, Nathanaël Fijalkow, and Karoliina Lehtinen. 2024. From Muller to Parity and Rabin Automata: Optimal Transformations Preserving (History) Determinism. *TheoretiCS* 3 (2024). doi:10.46298/THEORETICS.24.12

[7] Antonio Casares, Thomas Colcombet, and Karoliina Lehtinen. 2022. On the Size of Good-For-Games Rabin Automata and Its Link with the Memory in Muller Games. In *ICALP*, Vol. 229. 117:1–117:20. doi:10.4230/LIPIcs.ICALP.2022.117

[8] Thomas Colcombet. 2009. The Theory of Stabilisation Monoids and Regular Cost Functions. In *Automata, Languages and Programming, 36th Internatilonal Colloquium, ICALP 2009, Rhodes, Greece, July 5-12, 2009, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 5556)*, Susanne Albers, Alberto Marchetti-Spaccamela, Yossi Matias, Sotiris E. Nikoletseas, and Wolfgang Thomas (Eds.). Springer, 139–150. doi:10.1007/978-3-642-02930-1_12

[9] Thomas Colcombet. 2012. Forms of Determinism for Automata (Invited Talk). In *STACS (LIPIcs, Vol. 14)*. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 1–23. doi:10.4230/LIPICS.STACS.2012.1

[10] Daniele Dell'Erba, Yong Li, and Sven Schewe. 2024. DFAMiner: Mining Minimal Separating DFAs from Labelled Samples. In *FM (Lecture Notes in Computer Science, Vol. 14934)*. Springer, 48–66.

[11] Alexandre Duret-Lutz, Etienne Renault, Maximilien Colange, Florian Renkin, Alexandre Gbaguidi Aisse, Philipp Schlehuber-Caissier, Thomas Medioni, Antoine Martin, Jérôme Dubois, Clément Gillard, and Henrich Lauko. 2022. From Spot 2.0 to Spot 2.10: What's New?. In *CAV (Lecture Notes in Computer Science, Vol. 13372)*. Springer, 174–187.

[12] E. Mark Gold. 1978. Complexity of Automaton Identification from Given Data. *Inf. Control.* 37, 3 (1978), 302–320. doi:10.1016/S0019-9958(78)90562-4

[13] Rüdiger Ehlers and Ayrat Khalimov. 2024. A Naturally-Colored Translation from LTL to Parity and COCOA. *CoRR* abs/2410.01021 (2024). doi:10.48550/ARXIV.2410.01021

[14] Rüdiger Ehlers and Ayrat Khalimov. 2024. Fully Generalized Reactivity(1) Synthesis. In *TACAS*. Springer Nature Switzerland, 83–102.

[15] Rüdiger Ehlers and Sven Schewe. 2022. Natural Colors of Infinite Words. In *FSTTCS (LIPIcs, Vol. 250)*. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 36:1–36:17. doi:10.4230/LIPICS.FSTTCS.2022.36

[16] Javier Esparza, Orna Kupferman, and Moshe Y. Vardi. 2021. Verification. In *Handbook of Automata Theory*, Jean-Éric Pin (Ed.). European Mathematical Society Publishing House, Zürich, Switzerland, 1415–1456. doi:10.4171/AUTOMATA-2/16

[17] Javier Esparza, Rubén Rubio, and Salomon Sickert. 2024. Efficient Normalization of Linear Temporal Logic. *J. ACM* 71, 2 (2024), 1–42.

[18] Thomas A. Henzinger, Orna Kupferman, and Sriram K. Rajamani. 2002. Fair Simulation. *Inf. Comput.* 173, 1 (2002), 64–81. doi:10.1006/inco.2001.3085

[19] Thomas A. Henzinger and Nir Piterman. 2006. Solving Games Without Determinization. In *Computer Science Logic, CSL 2006 (Lecture Notes in Computer Science, Vol. 4207)*. Springer, 395–410. doi:10.1007/11874683_26

[20] Thomas A. Henzinger, Aditya Prakash, and K. S. Thejaswini. 2025. Resolving Nondeterminism with Randomness. In *MFCS (LIPIcs, Vol. 345)*. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 57:1–57:18.

[21] Denis Kuperberg and Michal Skrzypczak. 2015. On Determinisation of Good-for-Games Automata. In *ICALP (2) (Lecture Notes in Computer Science, Vol. 9135)*. Springer, 299–310. doi:10.1007/978-3-662-47666-6_24

[22] Orna Kupferman. 2018. Automata Theory and Model Checking. In *Handbook of Model Checking*, Edmund M. Clarke, Thomas A. Henzinger, Helmut Veith, and Roderick Bloem (Eds.). Springer International Publishing, 107–151. doi:10.1007/978-3-319-10575-8_4

[23] Orna Kupferman, Shmuel Safra, and Moshe Y. Vardi. 1996. Relating Word and Tree Automata. In *LICS*. IEEE Computer Society, 322–332. doi:10.1109/LICS.1996.561360

[24] Karoliina Lehtinen and Aditya Prakash. 2025. The 2-Token Theorem: Recognising History-Deterministic Parity Automata Efficiently. In *STOC*. ACM, 1839–1850.

[25] Christof Löding and Igor Walukiewicz. 2025. Minimal History-Deterministic Co-Büchi Automata: Congruences and Passive Learning. In *Symposium on Logic in Computer Science (LICS)*. 431–443. doi:10.1109/LICS65433.2025.00039

[26] Aditya Prakash. 2025. History-deterministic parity automata : games, complexity, and the 2-token theorem. (January 2025). http://webcat.warwick.ac.uk/record=b4197352 PhD thesis.

[27] Bader Abu Radi and Rüdiger Ehlers. 2025. Characterizing the Polynomial-Time Minimizable ω-Automata. *CoRR* abs/2504.20553 (2025). arXiv:2504.20553 doi:10.48550/ARXIV.2504.20553

[28] Bader Abu Radi, Orna Kupferman, and Ofer Leshkowitz. 2024. Easy Complementation of History-Deterministic Büchi Automata. In *ATVA 2024*, Vol. 15054. Springer, 67–88.

[29] Sven Schewe. 2010. Beyond Hyper-Minimisation—Minimising DBAs and DPAs is NP-Complete. In *IARCS Annual Conference on Foundations of Software Technology and Theoretical Computer Science, FSTTCS 2010, December 15-18, 2010, Chennai, India (LIPIcs, Vol. 8)*, Kamal Lodaya and Meena Mahajan (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 400–411. doi:10.4230/LIPIcs.FSTTCS.2010.400

A Proof of Lemma 3.5

Lemma 3.5. *The automaton $\mathcal{D}_{\mathtt{strong}}$ is a language-equivalent rewiring of $\mathcal{A}_{\mathtt{strong}}$.*

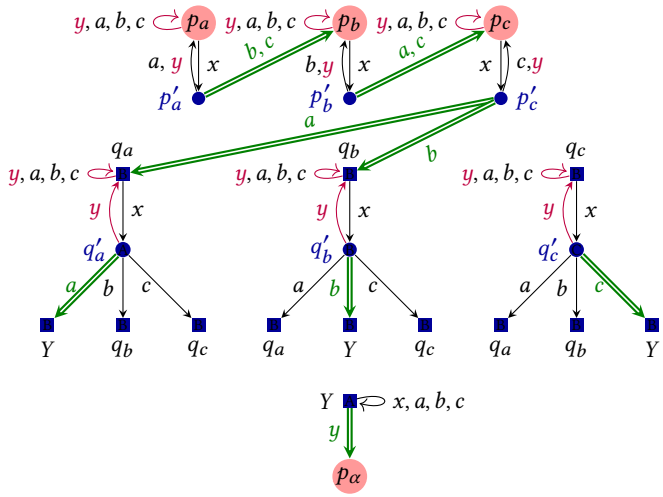


Figure 12: A language-equivalent rewiring $\mathcal{D}_{\mathtt{strong}}$ for $\mathcal{A}_{\mathtt{strong}}$. Redrawn from Fig. 5.

Proof. The fact that $\mathcal{D}_{\mathtt{strong}}$ is a rewiring of $\mathcal{A}_{\mathtt{strong}}$ is clear from the definition.

We first note that we can show that every word in $L_{\mathtt{strong}}$ is also in $L(\mathcal{D}_{\mathtt{strong}})$, similar to how we proved $\mathcal{A}_{\mathtt{strong}}$ is semantically deterministic in Lemma 3.3. Indeed, for every state q, $\mathtt{reach}(\mathcal{D}_{\mathtt{strong}}, q)$ and $\mathtt{reach}(\mathcal{A}_{\mathtt{strong}}, q)$ are structurally-isomorphic. For the other direction, we consider the following languages:

$$L_{\neq} = \bigcup_{\alpha \neq \beta \in \{a,b,c\}} L_\alpha L_\beta \ , \ L_{=} = \bigcup_{\alpha \in \{a,b,c\}} L_\alpha L_\alpha,$$

and

$$L_{\mathtt{Inf}(\neq)} = (\Sigma^* \cdot L_{\neq})^\omega \ , \ L_{\mathtt{Inf}(=)} = (\Sigma^* L_{=})^\omega \ , \ L_{\mathtt{Inf}(y)} = (\Sigma^* y)^\omega.$$

Then, note that

$$L_{\mathtt{strong}} = L_{\mathtt{Inf}(\neq)} \cap L_{\mathtt{Inf}(=)} \cap L_{\mathtt{Inf}(y)}.$$

We show that every word $w \in L(\mathcal{D}_{\mathtt{strong}})$ is indeed in each of these three languages.

For two states s, s' in $\mathcal{D}_{\mathtt{strong}}$, we let

$$L(s, s') = \{\text{Words that have a run from } s \text{ to } s'\}.$$

(1) $L(\mathcal{D}_{\mathtt{strong}}) \subseteq L_{\mathtt{Inf}(\neq)}$. This is true because every accepting run in $\mathcal{D}_{\mathtt{strong}}$ must visit p_a infinitely many times, as well as q_a or q_b infinitely many times. We observe that

$$L(p_a, q_a) \cup L(p_a, q_b) \subseteq \Sigma^* L_{\neq}$$

(2) $L(\mathcal{D}_{\mathtt{strong}}) \subseteq L_{\mathtt{Inf}(=)}$. Observe that this follows because

$$L(p'_c, Y) \subseteq \Sigma^* L_{=},$$

and every accepting run in $\mathcal{D}_{\mathtt{strong}}$ must visit p'_c and Y infinitely often.

(3) $L(\mathcal{D}_{\mathtt{strong}}) \subseteq L_{\mathtt{Inf}(y)}$. This follows because

$$L(Y, p_a) = \Sigma^* \cdot y,$$

and every accepting run in $\mathcal{D}_{\mathtt{strong}}$ must visit p_a and Y infinitely often.

We conclude that $L(\mathcal{D}_{\mathtt{strong}}) = L(\mathcal{A}_{\mathtt{strong}})$. □

B Proofs in Section 4

Theorem 4.2. *The automaton $\mathcal{A}_{\mathtt{main}}$ is history deterministic and recognises $L_{\mathtt{main}}$. Moreover, it is simplified.*

By Lemma 2.2, being a simplified Büchi automaton implies being history deterministic. Recall that a Büchi automaton is simplified if it is semantically deterministic, saturated, and has reach-covering.

Proposition B.1. *Every state in $\mathcal{A}_{\mathtt{main}}$ accepts the language $L_{\mathtt{main}}$ and $\mathcal{A}_{\mathtt{main}}$ is semantically deterministic.*

Proof. We remark that the language $L_{\mathtt{main}}$ is prefix-independent. Therefore, semantic determinism of $\mathcal{A}_{\mathtt{main}}$ follows from the first part of the claim.

We first show $L_{\mathtt{main}} \subseteq L(\mathcal{A}_{\mathtt{main}}, q)$. Let q be a state in $\mathcal{A}_{\mathtt{main}}$ and $w \in L_{\mathtt{main}}$. We inductively build an accepting run on w from q. At each step (except possibly the initialization step), we make sure that a significant transition is taken. Note that all significant transitions of $\mathcal{A}_{\mathtt{main}}$ end in some state in

$$Q_{\mathtt{signif}} = \{I\} \ \cup \bigcup_{\alpha} \{\bullet p_\alpha\} \ \cup \{Y\}.$$

As $L_{\mathtt{main}}$ is prefix-independent, every suffix of a word w in $L_{\mathtt{main}}$ also belongs to $L_{\mathtt{main}}$. Therefore, it suffices to show that every word $w \in L_{\mathtt{main}}$ admits a run reaching a significant transition from every state $q \in Q_{\mathtt{signif}}$, that is, $L_{\mathtt{main}} \subseteq L(\mathtt{reach}(\mathcal{A}_{\mathtt{main}}), q)$, for all $q \in Q_{\mathtt{signif}}$.

Initialization step. We show that a prefix of an arbitrary run reaches some state in $Q_{\mathtt{signif}}$. Note that w contains infinitely many y's. Therefore, upon reading letter y, a run reaches some state in

$$Q_{\mathtt{signif}} \cup \bigcup_{\alpha} \{\blacksquare p_\alpha\}.$$

If the run reaches $Q_{\mathtt{signif}}$ we are done. Assume that we reach a state $\blacksquare p_\alpha$ and let w' be the suffix of w from this point. Note that w' contains an infix in some L_β. Therefore, by Claim 4.1, it contains a prefix in $(L_{\beta'} + \Sigma^* \mathtt{r}_{\beta'})$; upon reading this prefix, we end up in $Q_{\mathtt{signif}}$.

Induction step. To build the run in the induction step, we distinguish three cases, according to the type of state. From Y, we see a significant transition upon reading y. From I, we notice that $L(\mathtt{reach}(\mathcal{A}_{\mathtt{main}}), I)$ contains the words in $\bigcup_\alpha (L_\alpha + \Sigma^* \mathtt{r}_\alpha)$. Since w contains an infix in some L_α, by Claim 4.1, the word w contains a prefix in this language.

The most challenging case is when $q = \bullet p_i$, for some i. Since $w \in L_{\mathtt{main}}$, it has a prefix of the form

$$u_0 u_\alpha u_\beta u'_\beta, \quad \text{with } u_\alpha \in (L_\alpha + \mathtt{r}_\alpha), \ u_\beta, u'_\beta \in L_\beta, \text{ and } \alpha \neq \beta.$$

Consider any run from $\bullet p_i$ over this word. We only need to show that some prefix of this run contains a significant transition. If there is no such significant transition seen, the run stays in the

(deterministic) blue box. If $u_\alpha = \mathsf{r}_\alpha$, the run over $u_0 u_\alpha$ finishes in $\bullet p_\alpha$. Since $\alpha \neq \beta$, it is clear that we have

$$\bullet p_\alpha \xrightarrow{u_\beta} \blacksquare p_\beta \xrightarrow{u'_\beta} \Rightarrow .$$

We focus therefore in the case $u_\alpha \in L_\alpha$. Note that, in this case, the last letter of u_α is α. After reading this letter, the automaton is necessarily in some state $\bullet p_\gamma$ or $\blacksquare p_\gamma$. From all these states, except from $\bullet p_\beta$, we reach a significant transition upon reading $u_\beta u'_\beta$. We argue that after reading u_α we cannot end up in $\bullet p_\beta$. Assume by contradiction that this is the case. Then, it must be the case that the two predecessors of $\bullet p_\beta$ in the run are in the β-box on the top row:

$$B_\beta = \{\bullet p_\beta, \bullet q_\beta, \bullet t_\beta, \bullet s_\beta, \bullet r_\beta\}.$$

We distinguish the cases $\alpha \in E$ and $\alpha \in F$. If $\alpha \in E$, then u_α ends by $c\alpha$. However, we have

$$B_\beta \xrightarrow{c} \{\bullet t_\beta, \bullet s_\beta\} \xrightarrow{\alpha} \bullet p_\alpha \neq \bullet p_\beta \ , \quad \text{a contradiction.}$$

If $\alpha \in F$, then u_α ends by $aA^*b\alpha$. However, we have

$$B_\beta \xrightarrow{aA^*} \{\bullet q_\beta, \bullet s_\beta, \bullet r_\beta\} \xrightarrow{b} \bullet r_\beta \xrightarrow{\alpha} \bullet p_\alpha \neq \bullet p_\beta,$$

a contradiction.

For the converse direction, let w be a word accepted by $\mathcal{A}_{\mathtt{main}}$. It is clear that it contains infinitely many y's, as an accepting run must go through Y infinitely often. To show that w contains infinitely many infixes in $(L_\alpha + \mathsf{r}_\alpha)L_\beta L_\beta$, we observe the following facts. By "consecutive occurrence" we mean that no state of type $\bullet p_\gamma$ or $\blacksquare p_\gamma$ appears in between.

- If u labels a path $I \xrightarrow{u} \Rightarrow \bullet p_\alpha$, then $u \in (L_\alpha + \Sigma^* \mathsf{r}_\alpha)$,
- If u labels a path between two consecutive occurrences of $\bullet p_\alpha$ and $\blacksquare p_\beta$, then $u \in L_\beta$ and $\alpha \neq \beta$,
- If u labels a path between two consecutive occurrences of $\blacksquare p_\alpha$ and $\blacksquare p_\beta$, then $u \in L_\beta$ and $\alpha \neq \beta$,
- If u labels a path between two consecutive occurrences of $\blacksquare p_\alpha$ and $\bullet p_\beta$, then $u \in \Sigma^* \mathsf{r}_\beta$,
- If u labels a path between two consecutive occurrences of $\blacksquare p_\beta$ and Y, then $u \in L_\beta$.

Consider a significant transition ending in Y on an accepting run over w in $\mathcal{A}_{\mathtt{main}}$. Then, w contains an infix in $(L_\alpha + \mathsf{r}_\alpha)L_\beta L_\beta$ for $\alpha \neq \beta$. This easily follows using the above properties, by considering the 3 last appearences of p-states or the state I, and doing a distiction of six cases according to whether these states are $\blacksquare p$, $\bullet p$ or I. □

Claim B.2. *$\mathcal{A}_{\mathtt{main}}$ is saturated.*

Proof. It suffices to observe that each rectangle in the image represents a strongly connected component induced by non-significant transitions. The rectangle containing the states I, I_a, I_b, and I_c forms one such strongly connected component, all the other states but Y forms another such strongly connected component, and finally the state Y alone also forms the final component. Any transitions between these three components are significant transitions. □

Proposition B.3. *$\mathcal{A}_{\mathtt{main}}$ has reach-covering.*

Proof. First, we note that the only non-determinism on $\mathcal{A}_{\mathtt{main}}$ is over the transition $I \xrightarrow{a}$. Therefore, the only states in $\mathcal{A}_{\mathtt{main}}$ that are not reach-deterministic are the 4 states in the top pink box: $B_{\mathtt{top}} = \{I, I_a, I_b, I_c\}$. For each of them, we find a reach-deterministic state that witnesses the definition of reach-covering. Since all states in $\mathcal{A}_{\mathtt{main}}$ are language-equivalent (by Claim B.1), the first condition of the definition of reach-covering is trivially satisfied, and it suffices to show that every state in $B_{\mathtt{top}}$ simulates some state outside $B_{\mathtt{top}}$. We show that I, I_a, I_b and I_c simulate, respectively, $\bullet p_1, \bullet q_1, \bullet r_1$ and $\bullet t_1$. The proof in the four cases is identical.

Consider the simulation game of $(\mathcal{A}_{\mathtt{main}}, \bullet p_1)$ by $(\mathcal{A}_{\mathtt{main}}, I)$. We describe a winning strategy for Eve. We denote the positions in this game (p_x, I_y); until one of the players reaches a significant transition, Adam's token remains in states in the big blue box, and Eve's token in states in $B_{\mathtt{top}}$. The strategy will maintain the following invariant:

If Adam's token is in some state $\blacksquare p_i$, then Eve's token is in I.

Actually, any possible strategy satisfies this invariant: Adam's token can only enter a state $\blacksquare p_i$ upon reading a letter in $E + F + y$. For all these letters, the transitions of Eve's token are of the form $B_{\mathtt{top}} \to I$ (or a significant transition).

The only positions where Eve has a choice to make is when her token is in state I and Adam gives letter a. We define Eve's choice in positions of the form $(\blacksquare p_i, I)$ over a. If $i \in E$, Eve stays in I, that is, she follows $I \xrightarrow{a} I$. If $i \in F$, Eve moves to the right, that is, she follows $I \xrightarrow{a} I_a$. Over all other positions of the game, we define Eve's choice arbitrary (note that in most of them she has no choice at all).

We show that this is a winning strategy. Assume that in a play, Adam reaches a significant transition. We show that Eve does too. Consider the last moment during the play when Adam's token is in a state of the form $\blacksquare p_i$ (one such a state necessarily occurs), and let u_i be the suffix of the word of the play from this moment. We have that $u_i \in L_i$, and moreover, $u_i \in A^* i$ (as $\blacksquare p_i$ is not visited again). Assume that $i \in E$ (Eve decides to stay on the left part of $B_{\mathtt{top}}$). If u_i does not contain letter a, then it is of the form $u_i = c^+ i$ (this is immediate from the transitions of the ith box in the bottom row of Figures 10a and 10b). Therefore, Eve sees the significant transition $I_c \overset{i}{\Rightarrow}$ at the same time as Adam. If u_i contains letter a, then it must be of the form $u_i = u_0 a u'_i c i$, with $u'_i \in (b + c)^*$. Then, Adam necessarily follows the path:

$$\blacksquare p_i \xrightarrow{u_0} \cdot \xrightarrow{a} \blacksquare q_i \xrightarrow{u'_i} \cdot \xrightarrow{c} \blacksquare s_i \overset{i}{\Rightarrow} .$$

On the left part of $B_{\mathtt{top}}$, Eve's token follows the path:

$$I \xrightarrow{u_0} \cdot \xrightarrow{a} I \xrightarrow{u'_i} \cdot \xrightarrow{c} I_c \overset{i}{\Rightarrow},$$

as wanted.

Finally, assume that $i \in F$ (Eve decides to go to the right part of $B_{\mathtt{top}}$). In this case, u_i must be of the form $(b + c) * aA^* bi$. Therefore, Eve's token follows the path

$$I \xrightarrow{(b+c)^*} \cdot \xrightarrow{a} I_a \xrightarrow{A^*} \cdot \xrightarrow{b} I_b \overset{i}{\Rightarrow},$$

and also produces a significant transition. □

We conclude that $\mathcal{A}_{\mathtt{main}}$ is simplified, finishing the proof of Theorem 4.2.

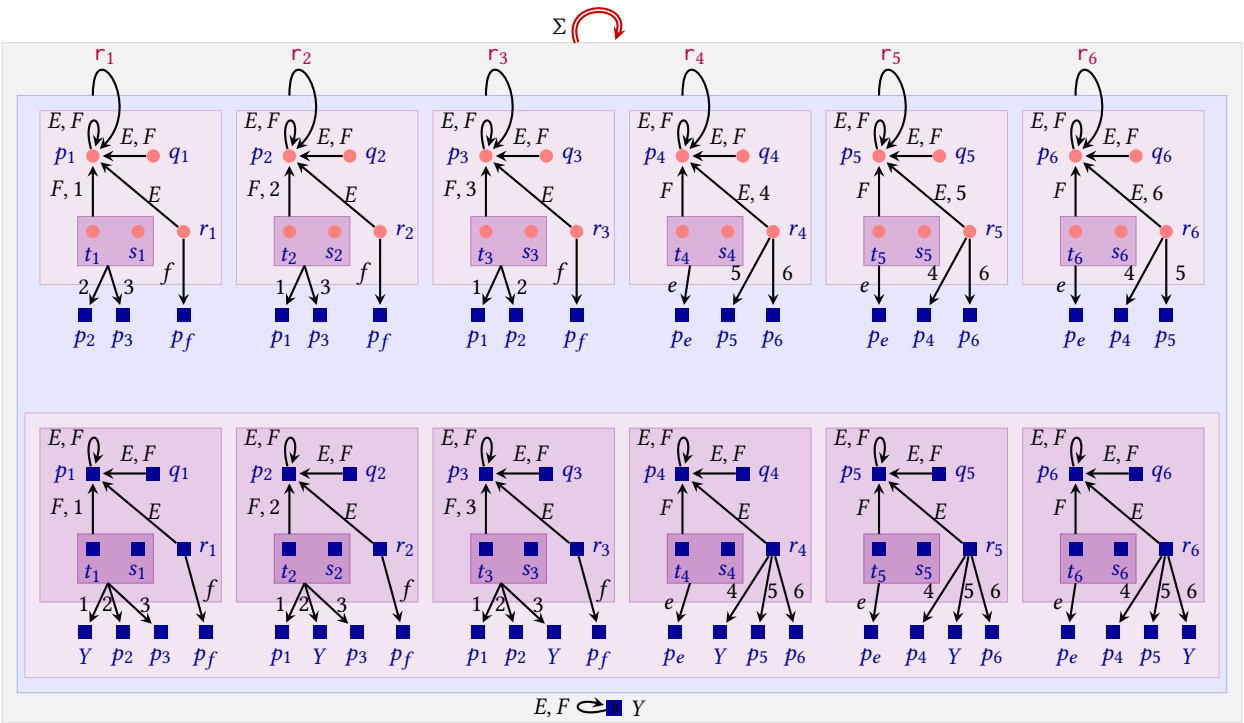


Figure 13: The HD coBüchi automaton obtained from $\mathcal{A}_{\mathsf{main}}$. The transitions over the alphabet $\{1, \ldots, 6\} \cup \{\mathsf{r}_1, \ldots, \mathsf{r}_6\}$. The transitions over the rest of the alphabet $\{a, b, c, y\}$ are similar to that in Fig. 10a. Similarly, we assume that $E = \{1, 2, 3\}$ and $F = \{4, 5, 6\}$and that e ranges from the set E and s from the set F

C Proofs in Section 5

Lemma 5.4. *The automata C_{main} is history-deterministic, and recognises the complement of the language recognised by $\mathcal{A}_{\mathsf{main}}$.*

Proof. We will prove that

(1) $L(C_{\mathsf{main}}) \subseteq L(\mathcal{A}_{\mathsf{main}})^c$,
(2) $L(\mathcal{A}_{\mathsf{main}})^c \subseteq L(C_{\mathsf{main}})$,
(3) and that there is an HD-resolver for C_{main}.

Proof of 1. Let w be a word in $L(C_{\mathsf{main}})$, and ρ be an accepting run of C_{main} on w. Then, there is a suffix u of w and a state p in C_{main} such that the (deterministic) run from p on u stays in the safe component containing p and does not contain a significant transition. Then, $u \in L_{\mathsf{safe}}(C_{\mathsf{main}}, p)$, and thus, $u \notin L(\mathsf{reach}(\mathcal{A}_{\mathsf{main}}), p)$. It is clear that $L(\mathcal{A}_{\mathsf{main}}, p) \subseteq L(\mathsf{reach}(\mathcal{A}_{\mathsf{main}}, q))$, and thus, it follows that $u \notin L(\mathcal{A}_{\mathsf{main}}, p)$. Since $L(\mathcal{A}_{\mathsf{main}}, p)$ is prefix-independent and $\mathcal{A}_{\mathsf{main}}$ is semantically-deterministic, $w \notin L(\mathcal{A}_{\mathsf{main}})$, as desired.

Proof of 2. Let $w = a_0 a_1 a_2 \ldots$ be a word that is not in $L(\mathcal{A}_{\mathsf{main}})$. We claim that there is a state p of $\mathcal{A}_{\mathsf{main}}$ and a suffix $w_i = a_i a_{i+1} \ldots$ of w such that every run of $(\mathcal{A}_{\mathsf{main}}, p)$ on w_i contains no significant transition. If this is not the case, then it is easy to inductively build a run of $\mathcal{A}_{\mathsf{main}}$ on w that contains infinitely many significant transitions, which contradicts the fact that $w \notin L(\mathcal{A}_{\mathsf{main}}, p)$.

Thus, $w_i \notin L(\mathsf{reach}(\mathcal{A}_{\mathsf{main}}), p)$ for some state p. We know, due to reach-covering for $\mathcal{A}_{\mathsf{main}}$ (Claim B.3), that there is a reach-deterministic state q such that p simulates q in $\mathsf{reach}(\mathcal{A}_{\mathsf{main}})$. Then,

$$L(\mathsf{reach}(\mathcal{A}_{\mathsf{main}}), p) \supseteq L(\mathsf{reach}(\mathcal{A}_{\mathsf{main}}), q),$$

and thus $w_i \notin L(\mathsf{reach}(\mathcal{A}_{\mathsf{main}}), q)$. This implies, by construction of C_{main}, that $w_i \in L_{\mathsf{safe}}(C_{\mathsf{main}}, q)$. Thus, $w \in L(C_{\mathsf{main}})$, since there is a run ρ of C_{main} on w that takes arbitrary transitions on $a_0 a_1 \ldots a_{i-1}$, takes the (possibly significant) transition to q on a_i, and then stays in the safe-component containing q by taking deterministic non-significant transitions.

3. C_{main} is HD. We next prove that C_{main} is HD, by describing an resolver γ for it. The resolver γ, after reading the prefix $u = a_0 a_1 \ldots a_{i-1}$, will be in some state q_i. On reading the letter a_i, γ takes the deterministic non-significant transition $q_i \xrightarrow{a_i} q_{i+1}$, if it exists. Otherwise, γ takes the significant transition $q_i \overset{a_i}{\Longrightarrow} p$ to a state p in C_{main}, where p is the state with the longest suffix v of ua_i such that there is a run of $\mathsf{safe}(C_{\mathsf{main}})$ on v that starts at some state p' and ends at p.

If the word w on which γ constructs a run is in $L(C_{\mathsf{main}})$, then there is a state p and a suffix w' of w for which $w' \in L(\mathsf{safe}(C_{\mathsf{main}}), p)$. Thus, the run of γ on w can only contain finitely many significant transitions before it eventually coincides with the deterministic run of (C_{main}, p) on w' consisting of only non-significant transitions. Thus, γ is an HD-resolver for C_{main}, as desired. □

Lemma 5.5. *The automaton C_{main} is the statewise smallest HD coBüchi automaton recognising $L(C_{\mathsf{main}})$.*

Proof. From Lemma 5.2, we need to prove that C_{main} is HD, semantically-deterministic, safe-deterministic, nice, safe-minimal, and safe-centralised. We argued that C_{main} is HD in Lemma 5.4,

$C_{\mathtt{main}}$ is SD because every pair of states has a transition from one to the other on every letter, and $C_{\mathtt{main}}$ is safe-deterministic and nice by construction. We thus need to argue that $C_{\mathtt{main}}$ is safe-centralised and safe-minimised.

Safe-centralised. We note that there are only two safe-components in $C_{\mathtt{main}}$: the safe component consisting $\mathcal{Y}$ of the state $\{Y\}$, and the safe component consisting of the other 60 states, which we call $\mathcal{S}^C_{\mathtt{safe}}$ due to its resemblance to a bus. Note that $L_{\mathtt{safe}}(C_{\mathtt{main}}, Y) = (\Sigma \setminus \{y\})^\omega$, where Σ is the alphabet of $C_{\mathtt{main}}$, while $y^\omega \in L_{\mathtt{safe}}(C_{\mathtt{main}}, s)$ for every state s in $\mathcal{S}^C_{\mathtt{safe}}$. Thus,

$$L_{\mathtt{safe}}(C_{\mathtt{main}}, Y) \not\subseteq L_{\mathtt{safe}}(C_{\mathtt{main}}, s)$$

for every state $s \in \mathcal{S}^C_{\mathtt{safe}}$.

Similarly, we note that for each state s in $\mathcal{S}^C_{\mathtt{safe}}$, there is a word $w \in (\Sigma \setminus \{y\})^*$ such that $w \notin L_{\mathtt{safe}}(C_{\mathtt{main}}, s)$, but w is in $L_{\mathtt{safe}}(C_{\mathtt{main}}, y)$. It follows that

$$L_{\mathtt{safe}}(C_{\mathtt{main}}, s) \not\subseteq L_{\mathtt{safe}}(C_{\mathtt{main}}, Y)$$

for every state $s \in \mathcal{S}^C_{\mathtt{safe}}$. Thus, $C_{\mathtt{main}}$ is safe-centralised.

Safe-minimised. We need to show that for every pair of distinct states $p', q' \in \mathcal{S}^C_{\mathtt{safe}}$, we have that $L_{\mathtt{safe}}(C_{\mathtt{main}}, p') \neq L_{\mathtt{safe}}(C_{\mathtt{main}}, q')$. We split the proof in the following 5 cases.

Case 1. $p' = \bullet\pi_i$ and $q' = \blacksquare\pi'_j$. Then, there is a finite word v in L_j for some $i \in [6]$, such that

$$yvy^\omega \in L(\mathtt{safe}(C_{\mathtt{main}}, \bullet\pi_i)) \setminus L(\mathtt{safe}(C_{\mathtt{main}}, \blacksquare\pi'_j)).$$

Case 2. $p' = \bullet\pi_i$ and $q' = \bullet\pi'_j$ for some $i \neq j$. Then, there is a word $v_j \in L_j$ such that

$$yv_jv_jy^\omega \in L_{\mathtt{safe}}(C_{\mathtt{main}}, \bullet p_i) \setminus L_{\mathtt{safe}}(C_{\mathtt{main}}, \blacksquare\pi'_j)$$

Case 3. $p' = \blacksquare\pi_i$ and $q' = \blacksquare\pi'_j$ for some $i \neq j$. Then, there is a finite word v_j in L_j such that

$$yv_jy^\omega \in L_{\mathtt{safe}}(C_{\mathtt{main}}, \bullet\pi_i) \setminus L_{\mathtt{safe}}(C_{\mathtt{main}}, \blacksquare\pi'_j)$$

Case 4. $p' = \bullet\pi_i, q' = \bullet\pi'_i$, for some $\pi \neq \pi' \in \{p, q, r, s, t\}$. Let $f \in F, s \in S$ be such that $f, s \neq i$. We have the following 5 subcases here.

Case 4a. $\pi \in \{p, t\}, \pi' \in \{q, r, s\}$. Then, the word cs is such that $\bullet\pi_i \xrightarrow{cs} \bullet p_i$, and $\bullet\pi'_i \xrightarrow{cs} \blacksquare p_s$. Since $L_{\mathtt{safe}}(C_{\mathtt{main}}, \bullet p_i) \neq L_{\mathtt{safe}}(C_{\mathtt{main}}, \blacksquare p_s)$ from Case 1, we obtain that $L_{\mathtt{safe}}(C_{\mathtt{main}}, \bullet\pi_i) \neq L_{\mathtt{safe}}(C_{\mathtt{main}}, \bullet\pi'_i)$.

Case 4b. $\pi = p, \pi' = t$. Then, $\bullet\pi_i \xrightarrow{f} \bullet p_i$, whereas $\bullet\pi'_i \xrightarrow{f} \blacksquare p_f$. $L_{\mathtt{safe}}(C_{\mathtt{main}}, \pi_1) \neq L_{\mathtt{safe}}(C_{\mathtt{main}}, \pi_2)$ then follows from Case 1.

Case 4c. $\pi = q, \pi' = r$. Then $\bullet\pi_i \xrightarrow{s} \bullet p_i$, whereas $\bullet\pi'_i \xrightarrow{s} \blacksquare p_s$. $L_{\mathtt{safe}}(C_{\mathtt{main}}, \pi_1) \neq L_{\mathtt{safe}}(C_{\mathtt{main}}, \pi_2)$ then follows from Case 1.

Case 4d. $\pi = q, \pi' = s$. Then, $\bullet\pi_i \xrightarrow{s} \bullet p_i$, whereas $\bullet\pi'_i \xrightarrow{s} \blacksquare p_s$. $L_{\mathtt{safe}}(C_{\mathtt{main}}, \pi_1) \neq L_{\mathtt{safe}}(C_{\mathtt{main}}, \pi_2)$ then follows from Case 1.

Case 4e. $\pi = r, \pi' = s$. Then, $\bullet\pi_i \xrightarrow{s} \blacksquare p_s$, and $\bullet\pi'_i \xrightarrow{s} \bullet p_i$. $L_{\mathtt{safe}}(C_{\mathtt{main}}, \pi_1) \neq L_{\mathtt{safe}}(C_{\mathtt{main}}, \pi_2)$ then follows from Case 1.

Case 5. $p' = \blacksquare\pi_i, q' = \blacksquare\pi'_i$, for some $\pi \neq \pi' \in \{p, q, r, s, t\}$. Let $f \in F, s \in S$ be such that $f, s \neq i$. We have the following subcases here.

Case 5a. $\pi \in \{p, t\}, \pi' \in \{q, r, s\}$. Then, the word cs is such that $\blacksquare\pi_i \xrightarrow{cs} \blacksquare p_i$, and $\blacksquare\pi'_i \xrightarrow{cs} \blacksquare p_s$. Since $L_{\mathtt{safe}}(C_{\mathtt{main}}, \blacksquare p_i) \neq L_{\mathtt{safe}}(C_{\mathtt{main}}, \blacksquare p_s)$ from Case 3, we obtain that $L_{\mathtt{safe}}(C_{\mathtt{main}}, \blacksquare\pi_i) \neq L_{\mathtt{safe}}(C_{\mathtt{main}}, \blacksquare\pi'_i)$.

Case 5b. $\pi = p, \pi' = t$. Then, $\blacksquare\pi_i \xrightarrow{f} \blacksquare p_i$, whereas $\blacksquare\pi'_i \xrightarrow{f} \blacksquare p_f$. $L_{\mathtt{safe}}(C_{\mathtt{main}}, \pi_1) \neq L_{\mathtt{safe}}(C_{\mathtt{main}}, \pi_2)$ then follows from Case 3.

Case 5c. $\pi = q, \pi' = r$. Then $\blacksquare\pi_i \xrightarrow{s} \blacksquare p_i$, whereas $\blacksquare\pi'_i \xrightarrow{s} \blacksquare p_s$. $L_{\mathtt{safe}}(C_{\mathtt{main}}, \pi_1) \neq L_{\mathtt{safe}}(C_{\mathtt{main}}, \pi_2)$ then follows from Case 3.

Case 5d. $\pi = q, \pi' = s$. Then, $\blacksquare\pi_i \xrightarrow{s} \blacksquare p_i$, whereas $\blacksquare\pi'_i \xrightarrow{s} \blacksquare p_s$. $L_{\mathtt{safe}}(C_{\mathtt{main}}, \pi_1) \neq L_{\mathtt{safe}}(C_{\mathtt{main}}, \pi_2)$ then follows from Case 3.

Case 5e. $\pi = r, \pi' = s$. Then, $\blacksquare\pi_i \xrightarrow{s} \blacksquare p_s$, and $\blacksquare\pi'_i \xrightarrow{s} \blacksquare p_i$.

$$L_{\mathtt{safe}}(C_{\mathtt{main}}, \pi_1) \neq L_{\mathtt{safe}}(C_{\mathtt{main}}, \pi_2)$$

then follows from Case 3.

Thus, $C_{\mathtt{main}}$ is safe-minimal, safe-centralised, semantically-deterministic, HD, and nice, and thus statewise minimal Lemma 5.2. □

For a language L given by a safety automaton, define $\mathsf{Fin}(L)$ to be the language over finite words that are prefixes of words in L. We also let P_i be the set of states $\{\bullet p_i, \bullet q_i, \bullet r_i, \bullet s_i, \bullet t_i\}$ in $C_{\mathtt{main}}$, for some $i \in [6]$.

Lemma 5.16. *Under Assumption 5.15, there are at least 6 states outside $\mathcal{D}_{\mathsf{core}}$.*

Proof. We instead prove the following technical proposition to prove the Lemma above. This automatically shows that there are at least six distinct states $\pi_1, \pi_2, \ldots, \pi_6$ apart from at least 60 states in $\mathcal{D}_{\mathsf{core}}$.

Proposition C.1. *Suppose Assumption 5.15 is true. Then, for each $i \in [6]$, there is a state π_i outside $\mathcal{D}_{\mathsf{core}}$ such that either:*

(1) *π_i is in $\mathcal{S}^{\mathcal{D}}_{\mathtt{safe}}$, and for every word*

$$uj \in \mathsf{Fin}(\mathcal{K}_y) \setminus \mathsf{Fin}(L_{y\mathtt{safe}}(\pi_i)),$$

$j \in [6]$, the run from π_i on u ends at d_i in $\mathcal{D}_{\mathsf{core}}$ with $L_{\mathtt{safe}}(\mathcal{D}_{\mathtt{main}}, d_i) = L_{\mathtt{safe}}(C_{\mathtt{main}}, p)$ for some $p \in P_i$.

(2) *π_i is in $\mathcal{D}_{\mathsf{core}}$, and for every word*

$$uy \in \mathsf{Fin}(\mathcal{K}) \setminus \mathsf{Fin}(L_{y\mathtt{safe}}(\pi_i)),$$

the run from π_i on uy ends at d_i in $\mathcal{D}_{\mathsf{core}}$ with $L_{\mathtt{safe}}(\mathcal{D}_{\mathtt{main}}, d_i) = L_{\mathtt{safe}}(C_{\mathtt{main}}, p)$ for some $p \in P_i$.

Proof. Fix $i \in [6]$, and assume to the contrary that there is no such π_i. We let D_i be the set of states in $\mathcal{D}_{\mathsf{core}}$ whose safe-languages are the same as one of the states in P_i. We observe that if q is not a state in D_i, then $v^3 \notin \mathsf{Fin}(L_{\mathtt{safe}}(\mathcal{D}_{\mathtt{main}}, q))$.

We construct a word $w \in L(C_{\mathtt{main}})$ and a rejecting run of $\mathcal{D}_{\mathtt{main}}$ on w, which will provide us the contradiction. Towards this, let q_0 be a state in $\mathcal{D}_{\mathsf{core}}$ that is not in D_i. Then, the run of $\mathcal{D}_{\mathtt{main}}$ from q_0 on v^3 contains a significant transition, and suppose that it reaches some state q_1. We let $w_0 = v^3$. We note that q_1 is not a state in D_i, since otherwise we can construct an

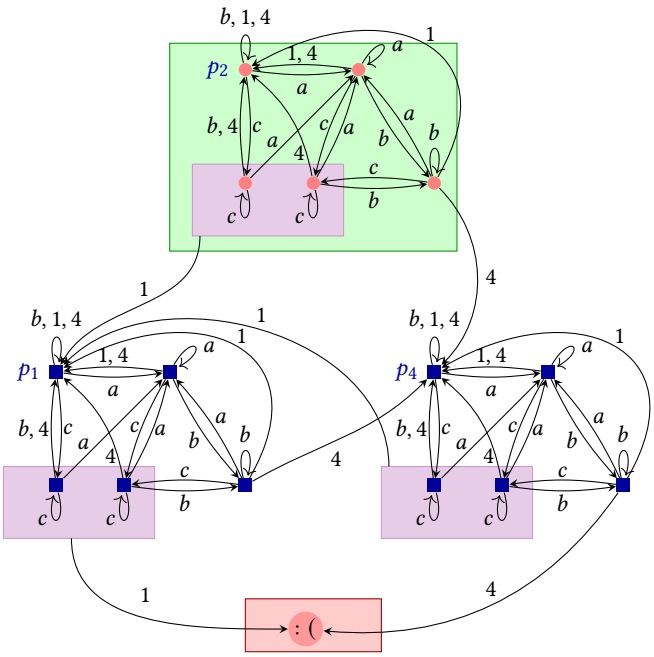


Figure 14: The 3-DFA for when q is $\bullet p_2$.

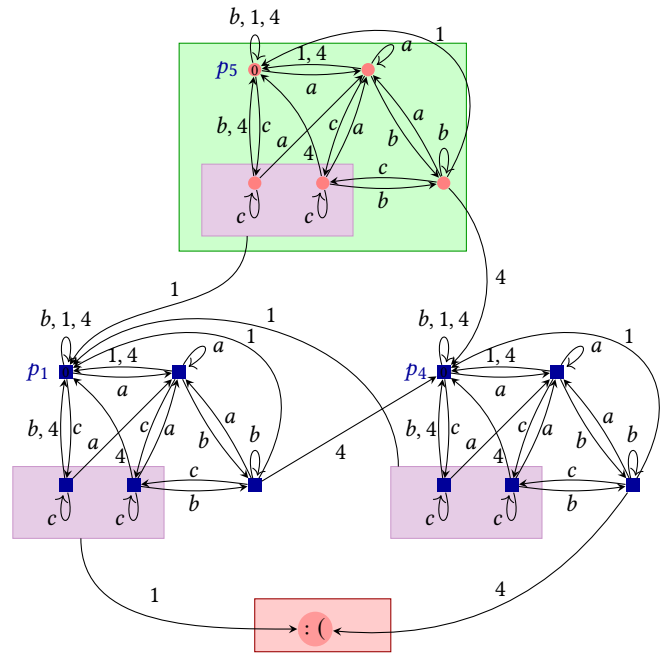


Figure 15: The 3-DFA for when q is $\bullet p_5$.

Suppose, so far, we have constructed a run on a finite word w_i from q_0 to q_i, where q_i is a state not in D_i, which we will ensure inductively. We have the following three cases:

Case 1. The state q_i is in $\mathcal{D}_{\text{core}}$. Then, we extend the word w_i by v^3 to obtain w_{i+1}. We note that $q_{i+1} \notin \mathcal{S}^{\mathcal{D}}_{\text{safe}}$: since $\mathcal{D}_{\text{core}}$ is a y-SCC, we can otherwise construct a rejecting run of $\mathcal{D}_{\texttt{main}}$ on a word that does not contain a y, which is a contradiction. Thus, $q_{i+1} \notin D_i$.

Case 2. The state q_i is in $\mathcal{S}^{\mathcal{D}}_{\text{safe}}$, but not in $\mathcal{D}_{\text{core}}$. Then, there is a word uj in $\mathcal{K}_y$ on which the run from q_i changes the y-SCC, and reaches a state q_{i+1} that is not in D_i. Note here that q_{i+1} would be in $\mathcal{S}^{\mathcal{D}}_{\text{safe}}$, since $\mathcal{K}_y \subseteq L_{\texttt{safe}}(\mathcal{D}_{\texttt{main}}, q_i)$. We extend the word w_i by uj to obtain w_{i+1}.

Case 3. The state q_i is not in $\mathcal{S}^{\mathcal{D}}_{\text{safe}}$. Then there is a word $uy \in \mathcal{K}$ such that the run of $\mathcal{D}_{\texttt{main}}$ from q_i on uy contains a significant transition and reaches a state q_{i+1} that is not in D_i. We extend the word w_i by uy to obtain w_{i+1}.

Define $\mathcal{K}_{num}$ to be the set of words in $\mathcal{K}$ that end in some j for $j \in [6]$.

Note that we cannot eventually have only Case 2 or 3 steps, since otherwise, we would produce a rejecting run of $\mathcal{D}_{\texttt{main}}$ on a word in $(\mathcal{K}_{num} + \mathcal{K}y)^\omega \in L(C_{\texttt{main}})$. Thus, we must have infinitely often Case 1 steps. But then, we produce a rejecting run of $\mathcal{D}_{\texttt{main}}$ on a word in $(L_i)^\omega \in L(C_{\texttt{main}})$, which is a contradiction. □

□

D Computer-aided lemmas

In this section, we will prove the following result by the aid of computers, namely, the tool DFAMiner [10].

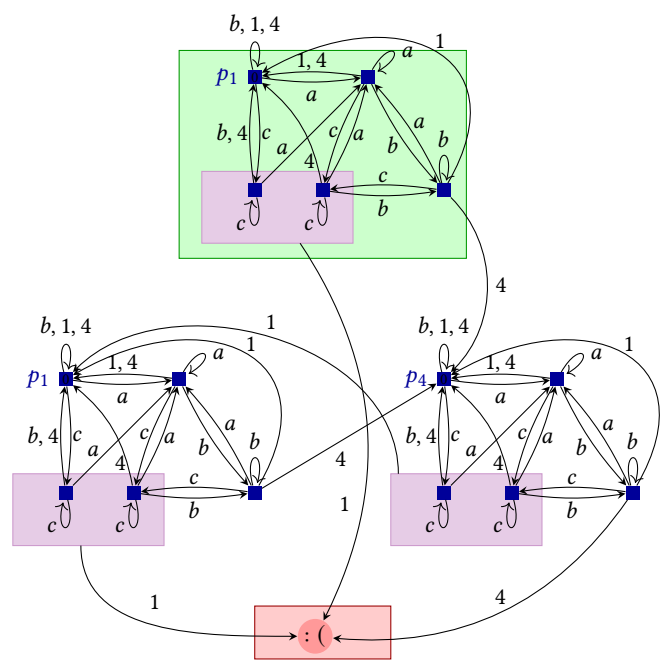


Figure 16: The 3-DFA for when q is $\blacksquare p_1$.

Lemma 5.12. *Let $\mathcal{D}$ be a deterministic safety automaton that separates $\mathcal{K}$ (resp. $\mathcal{K}_y$) and $L_{\texttt{safe}}(C_{\texttt{main}}, q)$ for some state q in $\mathcal{S}^{C}_{\text{safe}}$, i.e.,*

$$\mathcal{K} \subseteq L(\mathcal{D}) \subseteq L_{\texttt{safe}}(C_{\texttt{main}}, q).$$

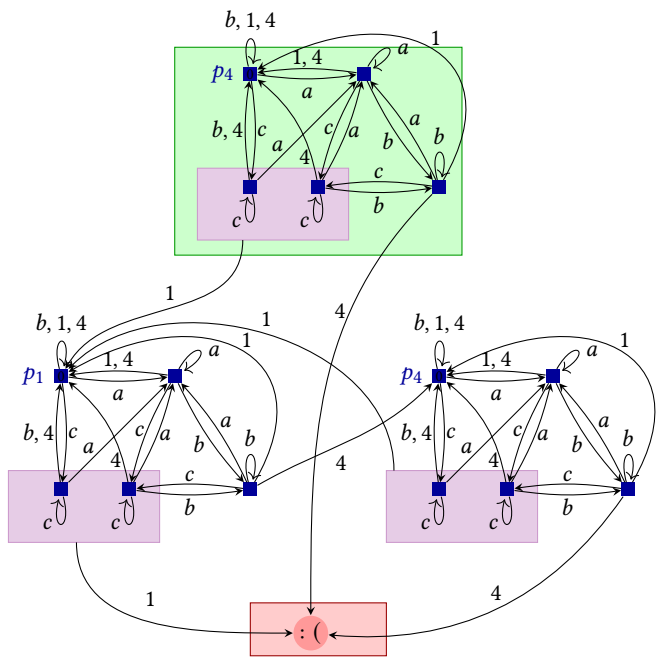


Figure 17: The 3-DFA for when q is $\blacksquare p_4$.

Then, $\mathcal{D}$ has at least 5 states.

We note that for two safety automata $\mathcal{A}$ and $\mathcal{B}$, $L(\mathcal{A}) \subseteq L(\mathcal{B})$ if and only if $\mathsf{Fin}(L(\mathcal{A})) \subseteq \mathsf{Fin}(L(\mathcal{B}))$. Thus, in this section, we view our safety automata as DFAs, where every state is accepting, together with an additional rejecting sink state for completion.

Towards proving Lemma 5.12, we will first restrict candidates for our state q in $\mathcal{S}^{C}_{\mathsf{safe}}$. Let d_0 be the initial state of $\mathcal{D}$. Let $q \xrightarrow{1} q'$, and $d_0 \xrightarrow{1} d'$. Note that $1 \equiv_L \varepsilon$ in $\mathcal{K}$ and $\mathcal{K}_y$. Thus, $(\mathcal{D}, d')$ separates $\mathcal{K}$ and $L_{\mathsf{safe}}(C_{\mathsf{main}}, q')$. But note that q' is either of the following states: $\bullet p_i$ for some $i \in [6]$, or $\blacksquare p_i$ for some $i \in [6]$. Furthermore, in $\mathcal{A}_{\mathsf{main}}$, the role of $\{1, 2, 3\}$ is symmetric, as well as $\{4, 5, 6\}$ is symmetirc. Thus, it suffices to prove Lemma 5.12 for when q' is a state in $\{\bullet p_2, \bullet p_5, \blacksquare p_1, \blacksquare p_4\}$.

For a DFA $\mathcal{A}$ over the alphabet Σ and a subset $\Gamma \subseteq \Sigma$, we use $\mathcal{A}^{\Gamma}$ to denote the DFA $\mathcal{A}$ whose transitions have been restricted to the alphabet Γ. We make another simplification, based on the following simple observation.

Proposition D.1. *Let $\mathcal{A}, \mathcal{B}$, and C be three DFA over Σ such that*

$$L(\mathcal{A}) \subseteq L(\mathcal{B}) \subseteq L(C).$$

Then, for every $\Gamma \subseteq \Sigma$, we have that

$$L(\mathcal{A}^{\Gamma}) \subseteq L(\mathcal{B}^{\Gamma}) \subseteq L(C^{\Gamma}).$$

For a language L over infinite words, we use L_{Γ} to denote the set of words in $L \cap \Gamma^{\omega}$. Due to Proposition D.1, it suffices to show the following result.

Lemma D.2. *Let $\Gamma = \{a, b, c, 1, 4\}$, and let $\mathcal{D}$ be a DFA that separates $\mathsf{Fin}(\mathcal{K}_{\Gamma})$ and $\mathsf{Fin}(L_{\mathsf{safe}}({C_{\mathsf{main}}}^{\Gamma}, q))$, for some $q \in \{\bullet p_2, \bullet p_5, \blacksquare p_1, \blacksquare p_4.\}$ Then, $\mathcal{D}$ has at least 6 states (accounting for a rejecting sink state).*

Note that in Lemma D.2, the size of $(C^{\Gamma}_{\mathsf{main}}, q)$ is 15 for each q, which is much smaller than 60. This allows us to use the tool DFAMiner.

D.1 Using DFAMiner

One of the features of DFAMiner is that it minimises 3-DFAs. A 3-DFA has the syntax of a DFA, but each state is labelled as either accept, reject, or don't care. The idea is that a 3-DFA $\mathcal{A}$ accepts a word w if the run of $\mathcal{A}$ on w ends at an accepting state, rejects w if the run of $\mathcal{A}$ on w ends at rejecting sink state.

The problem of 3-DFA minimisation is the following: Given a 3-DFA $\mathcal{A}$, construct the smallest 3-DFA $\mathcal{B}$ such that $\mathcal{B}$ accepts every word accepted by $\mathcal{A}$ and $\mathcal{B}$ rejects every word rejected by $\mathcal{B}$.

It is easy to see that the problem of separating regular languages can be reduced to the problem of minimising a 3-DFA, that is obtained by a product construction. For proving our Lemma D.2, because of the similarity of structure between the automaton recognising $\mathcal{K}$ and C_{main}, the resulting 3-DFAs all have 16 states for each candiate q, as shown in Figs. 14 to 17.

The tool DFAMiner, when asked to minimise these four 3-DFAs, outputs that the minimal 3-DFA has size 6, which coincides with the deterministic safety automaton recognising $\mathcal{K}_y$ together with a rejecting sink state. We attach an encoding of these automata in the desired format of the DFAMiner tool for readers to verify.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009